

Chapter 1: Computer Networks and the Internet

Internet definition (nuts and bolts view): A computer network interconnecting billions of devices worldwide.

Devices (end systems/hosts):

- Traditional: desktop PCs, Linux workstations, servers (store/transmit info like web pages, emails).
- Non-traditional: laptops, smartphones, tablets, TVs, gaming consoles, thermostats, appliances, cars, security systems, wearables, etc.

Communication structure:

- End systems are connected via communication links (coaxial cable, copper wire, optical fiber, radio spectrum).
- Links have transmission rates (bits/sec).
- Data transfer: sending system segments data + adds headers → packets → travel through network → reassembled at receiver.

Packet switches: Forward packets from incoming to outgoing links.

Types:

- Routers → network core. Using IP addresses
- Link-layer switches → access networks. Using MAC addresses

Route/Path = sequence of links & switches traversed by a packet.

ISPs (Internet Service Providers):

- Provide access via cable/DSL, LAN, WiFi (public places), or mobile wireless.
- Also connect content providers (websites, video servers).
- ISPs interconnect → form global Internet.
- Each ISP = independent network with routers, fiber links, IP protocol, naming/addressing rules

Protocols: Define rules for sending/receiving information.

- TCP + IP → most important, form TCP/IP.
- IP defines packet format.
- Protocols ensure interoperability.

Standards:

- Developed by IETF → documented as RFCs (Requests for Comments). Define protocols like TCP, IP, HTTP, SMTP. 7,000 RFCs exist.
- IEEE 802 Committee → defines Ethernet & WiFi standards.

Alternative view of Internet as a service: An infrastructure providing services to applications.

Applications:

- Traditional: email, web browsing.

- Modern: messaging, real-time maps (traffic), cloud music, video streaming, social networks, video conferencing, multiplayer games, location-based systems.
- All are distributed applications → involve multiple end systems exchanging data.
- Key point: Applications run on end systems, not packet switches. Switches only move data, not process apps.
- Internet provides programming interface to distributed applications:
 - “hooks” allowing sending/receiving apps to “connect” to, use Internet transport service
 - provides service options, similar to postal service

Protocol = rules for message exchange + actions.

Key: Protocols only work if both parties follow same rules.

Example:

Greeting “Hi” → reply “Hi” → proceed to ask time.

If reply is rude/irrelevant → no communication.

If no reply → stop asking.

Human Analogy 2 (class Q&A)

Teacher: “Any questions?” → Student raises hand → Teacher acknowledges → Student asks → Teacher answers.

Network Protocols

- Similar to human protocols, but entities are devices (computers, routers, phones).
- Govern all Internet activity:
- Low-level: flow of bits between two network cards.
- End systems: congestion control (rate of packet transmission).
- Routers: determine packet path.
- Protocols run everywhere in Internet.

Example (web request):

Client sends connection request to server.

Server replies with acceptance.

Client sends GET request for webpage.

Server responds with requested file.

Formal Definition: A protocol defines the format and order of messages exchanged between communicating entities, and the actions taken upon sending, receiving, or other events.

1.2 The Network Edge

End Systems (Hosts): Devices connected to the Internet, they sit at the edge of the Internet. Also called hosts, since they host (run) applications like web browsers, web servers, email clients, email servers.

- End systems = devices at network edge, PCs, smartphones, servers, IoT devices,
- Hosts = run applications (web browser, email client/server, etc.),
- Clients = desktops, mobile devices, laptops,
- Servers = powerful machines, store/distribute web pages, stream video, relay email,
- Data centers = large clusters of servers, e.g., Google with 50–100 data centers,

Internet of Things (IoT)

- Billions of devices connected to the Internet, sensors, wearables, home automation, toys,
- Benefits = automation, data collection, smart devices,
- Risks = security attacks, privacy breaches, hacked IoT devices, compromised personal data

Access Networks: Network connecting an end system to the first router (edge router) on the path to distant end systems.

Types of Networks:

1. Home Networks

- **Definition:** A small-scale network within a household that connects devices (PCs, phones, smart appliances) to each other and the Internet.
- **How it works:** Uses a home router, Wi-Fi, and a broadband connection (e.g., DSL, cable, fiber).
- **Typical Technologies:**
 - Wi-Fi (802.11b/g/n/ac): 11 Mbps to 1 Gbps
 - Ethernet (wired): 100 Mbps to 1 Gbps

2. Enterprise Access Networks

- **Definition:** Networks in companies, universities, and institutions that connect users and servers to the Internet.
- **How it works:**
 - Enterprise connects to an ISP via an institutional router.
 - Inside, devices connect via Ethernet switches (wired) and Wi-Fi access points (wireless).
 - Internal servers (e.g., mail, web) connect through the same network.
- **Technologies Used:**
 - Ethernet: 100 Mbps, 1 Gbps, 10 Gbps
 - Wi-Fi: 802.11b/g/n → 11 Mbps, 54 Mbps, 450 Mbps

3. Data Center/Wide Area Access Networks

- **Definition:** Large-scale networks connecting hundreds to thousands of servers inside data centers to each other and to the Internet.

- **Features:**
 - Extremely high bandwidth: 10s–100s of Gbps
 - Fast communication between servers for cloud services, storage, and applications
 - Outward connections to ISPs, enterprise networks, home networks, and mobile networks
- **Example:** Used by content providers like Google, Amazon, Netflix to deliver global services.

Network Access Technologies:

1. Wired: DSL (Digital Subscriber Line):

- **What it is:** Broadband Internet delivered over existing telephone lines (twisted-pair copper).
- **How it works:** Uses higher frequencies of the copper telephone line for data while keeping lower frequencies free for voice calls. Frequency-Division Multiplexing separates channels,
- **DSL Modem:** converts digital data → high-frequency tones for telephone wires; DSLAM at central office converts tones back to digital.
- **Dedicated access:** each home gets its own line to the DSLAM (not shared like cable).
- **Asymmetric speeds:**
 - Downstream: 24–52 Mbps
 - Upstream: 3.5–16 Mbps
- Higher download than upload to match user demand
- **Limits:** distance to CO (5–10 miles), interference, line gauge.

2. Wired: Cable Internet (HFC – Hybrid Fiber Coax):

- **What it is:** Broadband Internet service delivered over the same coaxial cables used for cable TV, combined with fiber in the backbone (Hybrid Fiber-Coax).
- **HFC (Hybrid Fiber-Coax):** fiber backbone connects to coaxial cables that reach individual homes. Fiber to neighborhood, coaxial cable to homes,
- **FDM (Frequency Division Multiplexing):** separates TV, Internet, and other channels into different frequency bands on the same cable.
- **Shared access:** many homes in a neighborhood share the same coax cable to the ISP's headend; heavy use can affect speeds.
- **Asymmetric speeds:** downstream (download) faster than upstream (upload).
 - Downstream: 40 Mbps – 1.2 Gbps
 - Upstream: 30–100 Mbps

3. Wired: Ethernet

- LAN technology, twisted-pair copper wires, Ethernet switches,
- Speeds = 100 Mbps, 1 Gbps, servers up to 10 Gbps,

Wireless: Shared wireless connection that links end systems (phones, laptops, etc.) to a router through a base station (access point: A device that connects wireless clients to a wired network, enabling Wi-Fi access.).

4. Wireless: Wireless LANs (WLANs):

- Used within or around a building (~100 ft range).
- Based on IEEE 802.11 WiFi standards (b/g/n).
- Transmission speeds: 11 Mbps → 54 Mbps → up to 450 Mbps.

5. Wireless: Wide-Area Cellular Networks:

- Provided by mobile operators.
- Coverage: 10s of kilometers (large areas).
- Transmission rates: tens of Mbps (**4G**), **higher with 5G**.

Host Sending Function & Packet Transmission

- **Host:** End system (e.g., computer, phone) that sends data into the network.
- **Process:**
 - Takes application message.
 - Breaks it into smaller pieces called packets (size = L bits).
 - Sends packets into the access network.
- **Transmission rate (R):**
 - Also called link capacity or link bandwidth.
 - Measured in bits per second (bps).
 - Packet transmission delay: Time to push an L-bit packet onto a link with rate R.
 - Formula: **Delay** = L/R

Links: Physical Media

- General idea: A bit travels between transmitter & receiver over a link.
 - Guided media: Signal travels inside a solid medium (copper, fiber, coax).
 - Unguided media: Signal travels freely through air (radio).

1. Twisted Pair (TP)

- Two insulated copper wires twisted together.
- Why twisted pair uses two cables: One carries the signal; the other provides a return path and helps cancel electromagnetic interference through twisting.
- Speeds:
 - Cat 5: 100 Mbps – 1 Gbps Ethernet
 - Cat 6: up to 10 Gbps Ethernet

2. Coaxial Cable

- Two concentric copper conductors, shielding reduces interference
- Bidirectional (data both ways).
- Broadband: Multiple frequency channels per cable.
- Speed: 100's of Mbps per channel.

3. Fiber Optic Cable

- Glass fiber carrying light pulses (each pulse = 1 bit). Pulses of light, very high speeds (tens/hundreds Gbps), long distance, immune to interference, hard to tap,
- High-speed: 10's–100's Gbps, point-to-point.
- Low error rate:
 - Repeaters spaced far apart.
 - Immune to electromagnetic noise.

4. Wireless Radio

- No physical wire; uses electromagnetic spectrum.
- Broadcast, half-duplex (one-way at a time).
- Challenges: Reflection, obstruction, interference/noise.
- Types:
 - WiFi (WLAN): 10–100's Mbps; range ~10's of meters.
 - Cellular (e.g., 4G): 10's Mbps; range ~10 km.
 - Bluetooth: Short-range, limited speed, cable replacement.
 - Terrestrial microwave: Point-to-point; ~45 Mbps per channel.
 - Satellite: Up to 45 Mbps per channel; long delay (~270 ms end-to-end).

1.3 The Network Core

Definition: Mesh of interconnected routers; packet-switched network forwards packets hop-by-hop across links.

Core Functions

- Forwarding (Switching):
 - Local action.
 - Moves arriving packet from router's input → correct output link.
 - Uses destination address in packet header.
- Routing:
 - Global action.
 - Determines source → destination path.
 - Based on routing algorithms.

Packet Switching

- **Uses store and forward transmission:** breaks application messages into packets, each packet carries header information (source, destination, sequencing), packets travel independently through the network via links and packet switches, packets are reassembled at the destination.
- Packets: L bits long.
- Link transmission rate: R bps.
- Transmission delay = L / R seconds.
- Rule: Router must receive entire packet before forwarding to next link.
- Example: L = 10 Kbits, R = 100 Mbps → delay = 0.1 msec.
- **Forwarding & Routing:** each packet has destination IP, routers use forwarding tables mapping destinations → outbound links, tables set automatically by routing protocols, protocols find shortest paths, dynamic updates.

Queueing in Packet Switching

- Why it happens: If packets arrive at a link faster than the link can send them.

- Effects:
 - Packets wait in a queue before transmission.
 - If router buffer (memory) is full → packets dropped (loss).
 - Condition: Queueing & loss occur when arrival rate > transmission rate.
- Store-and-forward pros/cons: Reliable since the whole packet is checked before forwarding, concurrent order forwarding, but adds latency.
- Direct switching pros/cons: Low latency as bits are forwarded immediately, but errors may propagate.

Circuit Switching

- What it is: Alternative to packet switching, reserves end-to-end resources (bandwidth, buffer, processing) for the duration of a call/session. A dedicated communication path (with fixed bandwidth) is established between sender and receiver for the entire duration of a session, just like a traditional telephone call, no sharing while idle → wasteful if silence or inactivity periods occur.
- Key features:
 - Dedicated resources on each link.
 - No sharing → guaranteed performance (like a private lane).
 - If the call is idle, resources stay unused (wasteful for bursty traffic).
 - Historically used in traditional telephone networks.

Circuit Switching Methods

- **FDM (Frequency Division Multiplexing):**
 - Link bandwidth divided into narrow frequency bands.
 - Each call gets its own band.
 - Call transmits at max rate of that band continuously.
- **TDM (Time Division Multiplexing):**
 - Link bandwidth divided into time slots.
 - Each call gets periodic slots.
 - Call transmits at max rate of the full channel, but only during its slot(s).
 - transmission rate = frame rate × bits per slot.

Numerical Example: 640,000 bits file, 1.536 Mbps link, 24 TDM slots → 64 kbps per slot, file transfer time = 10 s + 0.5 s setup = 10.5 s total.

Aspect	Circuit Switching	Packet Switching
Definition	Dedicated path/resources reserved end-to-end for the session	Data split into packets, each sent independently and shared over the network
Advantages	- Predictable performance - No queueing delay - Fixed rate per connection - Simple once set up	- Efficient for bursty data (statistical multiplexing) - No idle waste - Many users can share - No setup delay

Disadvantages	- Inefficient for bursty data (idle time wasted) - Call setup delay - Fewer users supported	- Variable delay & jitter - Possible packet loss - Needs extra protocols (for reliability, congestion control)
----------------------	---	--

Real-time apps (voice/video) prefer guaranteed bandwidth → circuit switching historically better for constant rate, packet switching improving with QoS.

Virtual circuit switching: Pre-establishes a logical path for all packets, combining circuit reliability with packet-switching flexibility.

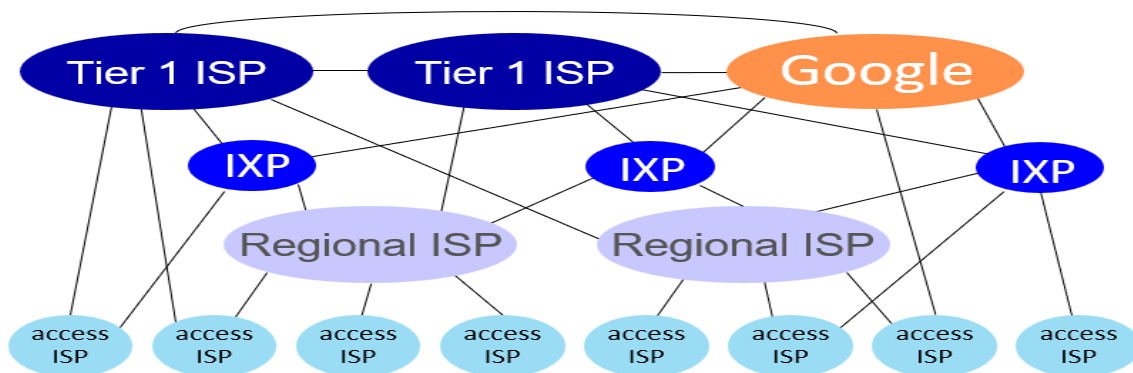
Constant bit rate vs. variable (bursty) bit rate: Constant bit rate sends data at a fixed rate; variable bit rate adjusts the rate based on traffic needs.

Internet Structure – “Network of Networks”

- Access ISPs: connect end users → Internet, use DSL, cable, FTTH, Wi-Fi, cellular, universities/companies can be access ISPs.
- Hosts connect to Internet via access ISPs.
- Access ISPs must interconnect so any two hosts can exchange packets.
- Internet is a complex network of networks, shaped by economics and national policies.

Stepwise Structure:

1. Home network → connects to local/regional ISP → connects to national/global ISP → connects to Internet backbone.
2. Direct interconnection between all access ISPs is impractical ($O(N^2)$ scaling).
3. **Structure 1:** single global transit ISP, all access ISPs connect, costly, single provider., economic agreements govern relationships.
4. **Structure 2:** multiple global transit ISPs, access ISPs choose provider, global ISPs interconnect for full reachability.
5. **Structure 3:** regional ISPs between access ISPs & tier-1 ISPs, multi-tier hierarchy, customer-provider relationships at each level. Competition: multiple global ISPs exist; regional networks arise to connect access ISPs efficiently.
6. Content provider networks (e.g., Google, Microsoft, Akamai) run private networks to bring content closer to end users, sometimes bypassing tier-1 ISPs.



Key Points at the Internet Core:

- Content provider networks: connect data centers to Internet, optimize latency and bandwidth, often bypassing tier-1 ISPs.
- Tier-1 ISPs: small number of large, well-connected networks, top-level providers (AT&T, NTT, Level 3, Sprint), global coverage, settlement-free peering with each other.
- PoPs (Points of Presence): routers at provider locations, customer ISPs connect via high-speed links, present in all levels except access ISPs.
- Multi-homing: ISP connects to multiple providers for redundancy, reliability, cost optimization.
- Peering: two ISPs at same level interconnect directly, exchange traffic settlement-free, avoids upstream provider costs.
- IXPs (Internet Exchange Points): facilities where multiple ISPs peer together, >400 IXPs worldwide, reduce costs, improve performance.

1.4 Delay, Loss, and Throughput in Packet-Switched Networks

Packet Delay and Loss

- **Cause of packet loss:**
 - Packets queue in **router buffers** waiting for transmission.
 - Queue length grows if **arrival rate > output link capacity**.
 - **Packet loss** occurs when buffer memory fills; excess packets are dropped.
- **Packet delay sources (nodal delay):** delay a packet experiences at a single router (node)

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

1. **Processing delay (d_{proc}):**
 - Time to examine packet header, check for bit errors, decide outbound link, usually < **microseconds**.
 2. **Queueing delay (d_{queue}):**
 - Time packets wait in buffer for transmission, depends on **router congestion**.
 3. **Transmission delay (d_{trans}):**
 - Time to push packet onto link.
 - Formula: **$d_{\text{trans}} = L/R$**
 - L = packet length (bits)
 - R = link transmission rate (bps)
 4. **Propagation delay (d_{prop}):**
 - Time for signal to travel physical link from one node to the next.
 - Formula: **$d_{\text{prop}} = d/s$**
 - d = link length (m)
 - $s \approx 2 \times 10^8$ m/s (signal speed in medium)
- **Summary:**
 - Transmission delay: depends on packet size & link bandwidth (L , R).

- Propagation delay: depends on physical distance & signal speed (d , s).
- Assume processing delay zero unless given

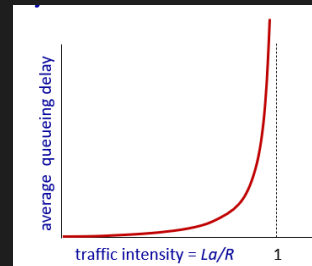
Caravan Analogy (Packet Transmission Analogy)

- **Setup:**
 - Caravan = packet (e.g., 10-bit packet)
 - Car = bit
 - Toll booth = link/router
 - Highway = physical link
 - Transmission = toll service; propagation = travel along highway
- **Example 1:**
 - Caravan: 10 cars
 - Toll service per car: 12 sec $\rightarrow$ time to push entire caravan = $12 \times 10 = 120$ sec
 - Distance to 2nd booth: 100 km at 100 km/hr $\rightarrow$ propagation time = 1 hr
 - **Observation:** last car reaches 2nd booth after 62 min
- **Example 2 (Faster propagation, faster service):**
 - Propagation: 1000 km/hr
 - Toll service: 1 min per car
 - First car reaches 2nd booth in 7 min
 - **Observation:** first car arrives before entire caravan is serviced at first booth

Packet Queueing Delay

It measures how busy a link is: average demand (arrival bits/sec) relative to link capacity. La/R .

- **Definitions:**
 - a = average packet arrival rate
 - L = packet length (bits)
 - R = link bandwidth (bps)
 - La/R = traffic intensity
- **Traffic intensity effects:**
 - $La/R \approx 0$: average queueing delay small
 - $La/R \rightarrow 1$: average queueing delay large
 - $La/R > 1$: arrival rate exceeds service rate $\rightarrow$ queue grows infinitely
- **Summary:**
 - Queueing delay increases as link becomes more congested
 - Traffic intensity is key factor for performance



Real Internet Delays and Routes

- **Traceroute tool:** measures delay from source to each router along end-to-end path
 - Sends three probe packets with TTL = i for router i
 - Router i returns packets to sender
 - Sender measures round-trip time for each probe

- **Example:** gaia.cs.umass.edu → www.eurecom.fr
 - Shows multiple hops, delays in ms, occasional missing replies
 - Missing replies (e.g., *******) indicate packet loss or routers not responding
 - Delays can fluctuate due to queuing, routing, or network load
- **Observation:** delays may not always increase monotonically; sometimes they decrease due to routing changes or measurement artifacts
- **End-to-End Delay:** sum of all nodal delays across all routers along path from source to destination.

1.4.3 End-to-End Delay

- For N links between source & destination, no queuing:

$$d_{end-end} = N(d_{proc} + d_{trans} + d_{prop})$$

- $d_{trans} = L/R$ for each link.

Traceroute

- Tool to measure path & round-trip delays to each router.
- Source sends N packets → each router returns its name, address, delay time.
- Shows variability in delays (queuing delay changes over time).

Traceroute Questions and Answers

A. If traceroute command runs again, will the IP address shown by point A always be the same for google.com?

- Answer: No. Google.com uses many servers with multiple IPs, and any one can respond. Google also uses anycast routing, so different servers may reply depending on network conditions.

B. What does '30 hops' represent?

- Answer: It is the maximum number of hops (routers) traceroute will probe before stopping. If the destination is farther than 30 hops, traceroute ends without success.

C. Which device is represented by the IP address of Point C?

- Answer: It represents the first router (gateway) on the path, usually the local router of the host's network.

D. What does * represent, as shown by point E?

- Answer: A * means no reply was received within the timeout for that probe.

E. Why does traceroute always show 'Request timed out' at point E?

- Answer: Because that router/device is configured to block ICMP echo replies, so it does not respond to traceroute probes.

F. What do the three values represent at points F or G, and why are they different?

- Answer: They are the Round Trip Times (RTTs) of three separate probes sent to the same hop. They often differ due to variable queuing delays. Multiple values are taken instead of one to estimate average performance.

G. Why are the values of F (hop 7) mostly smaller than G (hop 8)?

- Answer: Because hop 8 is farther in the network path, adding more transmission and propagation delay, while also possibly facing more queuing.

H. What is the importance of the number 13 at point H?

- Answer: It represents the final hop, i.e., the destination server was reached in 13 hops.

I. What is the URL shown at point I?

- Answer: It is the canonical (real) name of the server corresponding to the IP address, resolved via reverse DNS.

```
C:\Users\Public>tracert google.com
```

```
Tracing route to google.com [172.217.19.174]
over a maximum of 30 hops:
```

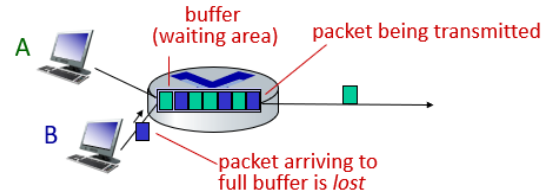
1	1 ms	1 ms	1 ms	192.168.43.1	C
2	*	*	*	Request timed out.	E
3	35 ms	49 ms	44 ms	10.81.77.165	
4	51 ms	36 ms	49 ms	10.81.73.137	
5	179 ms	50 ms	47 ms	10.180.76.1	
6	40 ms	75 ms	70 ms	10.81.171.21	
7	522 ms	67 ms	138 ms	116.71.129.225	F
8	60 ms	36 ms	40 ms	10.253.4.44	G
9	58 ms	78 ms	65 ms	10.253.4.8	
10	89 ms	86 ms	90 ms	74.125.118.170	
11	79 ms	55 ms	58 ms	172.253.51.205	
12	57 ms	56 ms	61 ms	209.85.250.17	
13	56 ms	57 ms	55 ms	zrh04s07-in-f174.1e100.net	H

Trace complete.

I

Packet Loss

- **Cause:** queue/buffer before link has finite capacity
 - Packet arriving at full buffer is dropped (lost)
 - Lost packets may be retransmitted by previous node, source end system, or may not be retransmitted at all
- **Illustration:**
 - A = packet being transmitted
 - B = packet waiting in buffer
 - C = packet arriving to full buffer → lost
- **Key points:**
 - Queueing precedes transmission → congestion leads to packet loss
 - Loss affects throughput and may trigger retransmissions (TCP)



Throughput

- **Definition:** rate (bits/time) at which bits are sent from sender to receiver
 - **Instantaneous:** rate at a given moment
 - **Average:** rate over longer period
- **Conceptual analogy:** server sends bits like fluid into a pipe
 - Pipe can carry bits at link capacity R (bits/sec)
- **End-to-end throughput:** determined by **bottleneck link**
 - **Bottleneck link:** link on end-to-end path with smallest capacity, constraining throughput
 - If sender rate $R_s < R_c$ (capacity), end-to-end throughput = R_s
 - If sender rate $R_s > R_c$, end-to-end throughput = R_c
- **Scenario:** When multiple flows (connections) share the same path, they must share the bandwidth of the bottleneck link (the slowest link in the path).
- **Formula:** Per-connection throughput $\approx \min(\text{sender rate, bottleneck link capacity} \div \text{number of connections})$
- **Key point:** network throughput is limited by **slowest (bottleneck) link** along the path

1.5 Protocol Layers and Their Service Models

- Internet = extremely complex system, many applications, protocols, end systems, packet switches, link-level media,
- Need structured organization → solution: layered architecture, modular design simplifies understanding, updates, teaching, maintenance.

Layering Concept & Analogy

- **Airline analogy:** ticketing → baggage → gate → runway → routing → landing → baggage claim → complaints,
- Each stage = one layer, provides service to layer above, uses service of layer below, Performing actions within that layer (e.g., gate layer loads/unloads passengers),
- Changing implementation of a layer without changing service → no effect on other layers, (e.g., boarding procedure change does not affect routing).

- Example: layer n can provide reliable delivery using unreliable delivery from layer n-1 by adding retransmission, error detection
- **Advantages:** modularity, abstraction, independent updates, clear structure, debugging ease, standardization (RFC 3439),
- **Drawbacks:**
 - Function duplication across layers → e.g., error recovery at link + transport layers,
 - Cross-layer information needs (e.g., timestamps) break strict separation.

Internet Protocol Stack (5 Layers)

Application Layer: network applications, distributed across end systems not core, use application-layer protocols to exchange messages, examples → HTTP (web documents), SMTP (email), FTP (file transfer), DNS (name resolution/translation), applications run on end systems, protocols define message formats, rules, interactions, packet at this layer = message.

Transport Layer: process-to-process delivery, sits between application & network layer, provides delivery between application processes, TCP → connection-oriented, reliable, flow control, congestion control, segmentation of data into segments, UDP → connectionless, unreliable, no congestion or flow control, faster but no guarantees, packet at this layer = segment.

Network Layer: host-to-host delivery across networks, delivers datagrams from source host to destination host using unique addressing, includes IP protocol (mandatory, unifies Internet), routing protocols → OSPF, BGP, RIP for path selection, uses destination address like postal system uses mailing address, packet at this layer = datagram, often called IP layer because IP dominates Internet communication.

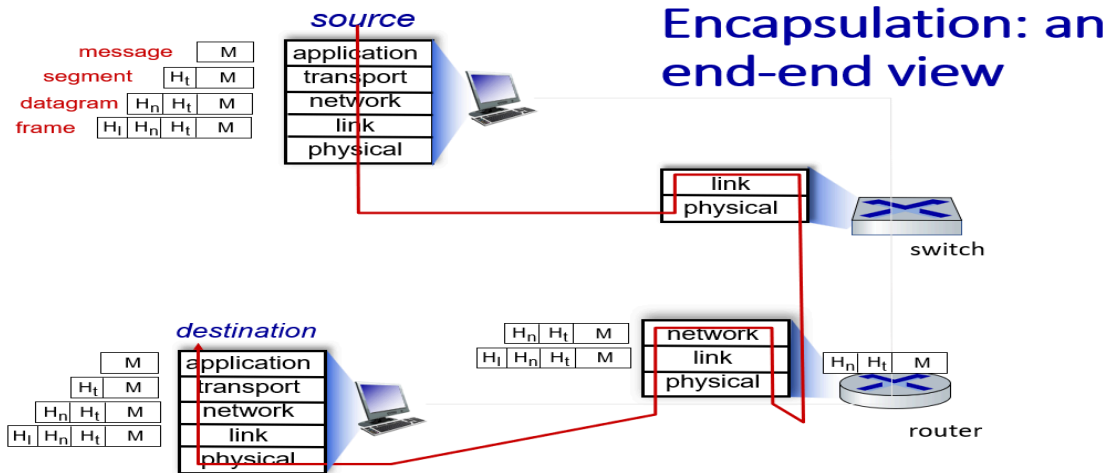
Link Layer: node-to-node delivery (host ↔ router, router ↔ router), moves datagram between adjacent network nodes, different links → different link-layer protocols, examples → Ethernet, WiFi, DOCSIS, PPP, some links offer per-link reliability (different from TCP's end-to-end reliability), packet at this layer = frame.

Physical Layer: transmission of raw bits within frames across physical medium, defines how bits are represented electrically/optically, protocols depend on transmission medium → twisted-pair copper, coaxial cable, fiber optics, wireless signals, multiple physical-layer standards exist even for same link-layer protocol (e.g., Ethernet over copper, fiber, coax).

Encapsulation & Data Units

- Each layer adds its own header around data from above:
 - Application data = Message,
 - Transport layer adds header → Segment,
 - Network layer adds header → Datagram,
 - Link layer adds header → Frame,
- Each layer packet = header + payload (payload = packet from above layer),

- **De-encapsulation** occurs at receiving side in reverse order,
- Large messages may be split into multiple segments/datagrams → reassembled at destination.
- Analogy: interoffice memo → envelope → postal package → delivery chain.



1. **Application Layer (HTTP):** Attaches the HTTP header containing fields like **URL/Host**, telling the server which web resource is requested.
2. **Transport Layer (TCP):** Adds the TCP header with **source/destination ports** (e.g., 80 for HTTP) and **sequence numbers**, enabling correct delivery to the application and ordered reassembly.
3. **Network Layer (IP):** Encapsulates into an IP datagram with **source/destination IP addresses**, ensuring end-to-end delivery across the Internet.
4. **Link Layer (Ethernet/Wi-Fi):** Adds the **frame header** with **source/destination MAC addresses**, used only for local hop-to-hop delivery (updated at each router).
5. **Physical Layer:** Converts the entire frame into **bits/signals** suitable for transmission over the physical medium (copper, fiber, wireless).

OSI vs Internet Stack

- OSI Model: 7 layers = Application, Presentation, Session, Transport, Network, Data Link, Physical,
- Extra OSI layers:
 - **Presentation:** encryption, compression, data format translation,
 - **Session:** synchronization, checkpointing, recovery,
- Internet model lacks these → left to application developers.

Layer Responsibilities & Implementation

- Application & Transport: software on end systems,

- Network: mix of software + hardware (e.g., routers),
- Link & Physical: NIC hardware + drivers,
- Switches implement Layers 1-2,
- Routers implement Layers 1-3,
- Hosts implement all 5 layers.

Layer Name	PDU	Device	Protocols	Addressing
Application	Message	PC / Laptop	HTTP, DNS	Specific address (URL)
Transport	Segment	Firewall/Gateway	TCP, UDP	Port address
Network	Datagram	Router	IP	Logical (IP) address
Data Link	Frame	Switch	Ethernet / PPP	Physical address

1.6 Network Security

Overview

- Internet not originally designed with strong security,
- Original vision: mutually trusting users on a transparent network,
- Designers now adding security after-the-fact (“catch-up”),
- Security considerations across all layers essential,
- Need to analyze: how attackers compromise networks, how to defend networks, how to design attack-resistant architectures.

Attack Types ("Bad Guys")

1. Packet Interception & Sniffing

- Packet sniffing: capturing packets on broadcast media (Ethernet, wireless),
- Promiscuous mode NIC reads all passing traffic (e.g., passwords),
- Tools: Wireshark for packet capture,
- Example: A → B → C, attacker reads packets destined for A.

2. Fake Identity (IP Spoofing)

- Injection of packets with false source IP,
- Attacker sends packet appearing to be from trusted source,
- Used in DoS, MITM, session hijacking attacks.

3. Denial of Service (DoS)

- Goal: make server/bandwidth unavailable to legitimate users,
- Steps: choose target → compromise multiple hosts → flood target with traffic,
- Often uses botnets for large-scale attacks.

Lines of Defense

- **Authentication:** proving user identity, SIM card in cellular networks, no hardware equivalent on Internet,

- **Confidentiality:** achieved via encryption,
- **Integrity:** digital signatures prevent/tamper detection,
- **Access restrictions:** VPNs, passwords, ACLs,
- **Firewalls:** middleboxes filtering packets by sender/receiver/app, off-by-default,
- **DoS detection:** filtering, rate limiting, anomaly detection

Chapter 2: Application Layer

Principles of Network Applications

- Applications run on end hosts, communicate over network using application layer protocols
- Network-core devices (routers, switches) don't run application software.
- End-system model enables rapid development and propagation of apps.

Types of Network Applications: Social media, web, text messaging, e-mail, multi-user games, streaming stored video (YouTube, Netflix, Hulu), P2P file sharing, VoIP, real-time video conferencing (Zoom), Internet search, remote login.

Application Layer: Focuses on conceptual and implementation aspects of network applications.

Application Architecture: The way an application is structured across various end systems is its application architecture.

Client-Server

- Server: always-on, permanent IP, often in data centers.
- Client: may connect intermittently, dynamic IP, requests server service.
- Communication is one-way from client to server, clients don't communicate with one another.
- Examples: HTTP, IMAP, FTP.

Peer-to-Peer (P2P)

- Peers directly communicate without dedicated servers. No always-on server; arbitrary end systems communicate directly. Peers request and provide services.
- Self-scalability: new peers add service capacity (by distributing) and workload (requesting) demand.
- Peers are intermittently connected; IP addresses change.
- Examples: BitTorrent.
- Pros: Self-scalable, cost-effective.
- Cons: Security, reliability, performance challenges due to their decentralized nature

Processes & Sockets

- **Processes vs Programs:** Processes are active instances of programs. A process is a running program on a host/end system
 - Same host → IPC (OS-defined).
 - Different hosts → message exchange over network
- In any communication session between two processes,
 - **Client process:** initiates communication.
 - **Server process:** waits to be contacted.

- Examples:
 - **Web:** Browser (client) ↔ Web server (server).
 - **P2P:** File downloader (client) ↔ File uploader (server).
 - **Dual Role in P2P:** A peer can be both client & server.

Sockets: A socket is the software interface that a process uses to send messages into and receive messages from the network. It's also known as the Application Programming Interface (API) between the application and the network.

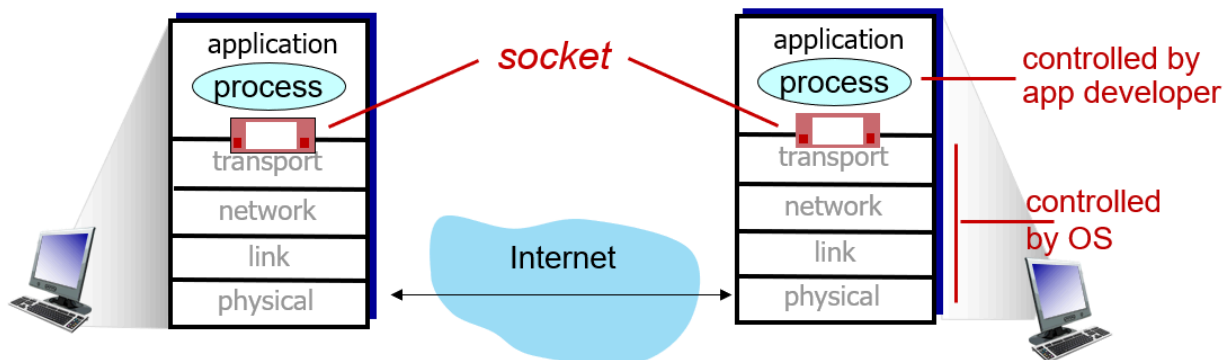
- **Analogy:** A process is like a house, and its socket is its door.
- **Layer Interface:** The socket is the boundary between the **application layer** and the **transport layer**. The application developer controls the application-layer side, but has limited control over the transport-layer side.
 - One socket on each side. Sending process pushes message into socket, transport delivers to receiving socket.
 - Controlled by OS & application developer.
 - Host IP alone is not enough (multiple processes per host). Two pieces of info needed to reach a process:
 - Host IP address (32 bit)
 - Port number (identifies process on host)

Example Ports:

HTTP server → port 80

Mail server → port 25

Send HTTP request: IP 128.119.245.12, port 80.



Application-Layer Protocols

- Define:
 - Message types (request, response)
 - Syntax (fields & format)
 - Semantics (meaning)
 - Rules (when/how to send)

Examples:

- **Open protocols (HTTP, SMTP, DNS)** → standardized, public application-layer protocols.

- **Proprietary protocols (Zoom, WhatsApp)** → closed, vendor-specific application-layer protocols.

Transport Services Needs for Applications

Transport protocol selection: Choice affects reliability, throughput, timing, security.

Transport Service Requirements:

Requirement	Examples
Data integrity	Guarantees error-free, complete delivery. (file transfer, web), some apps tolerate loss (audio).
Timing	Guarantees on delivery delay, low delay needed for telephony, games.
Throughput	Guarantees minimum rate, multimedia needs minimum throughput, elastic apps adapt to available throughput.
Security	encryption, integrity, authentication.

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video: 10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet Apps and Transport

- FTP → TCP,
- SMTP → TCP,
- HTTP → TCP,
- Internet telephony → TCP or UDP,
- Streaming audio/video → TCP (HTTP, DASH),
- Games → UDP or TCP.

Transport Protocols

- **TCP:** A reliable, connection-oriented, full-duplex protocol with flow control and congestion control, ensuring ordered and error-free delivery but offering no timing, throughput, or security guarantees; used when reliability matters (e.g., web, email, FTP, remote access).
- **UDP:** A lightweight, connectionless protocol without reliability, flow, or congestion control and no timing/throughput guarantees; suitable for loss-tolerant, low-delay applications like streaming, gaming, and DNS.

Vanilla TCP/UDP: Do not provide encryption, data (including passwords) sent in cleartext.

TLS (Transport Layer Security): adds encryption, integrity, endpoint authentication.

Implemented at application layer, TLS sockets → Encrypt data before sending via TCP.

Limitations of Internet Transport Protocols

- No throughput or timing guarantees (TCP/UDP).

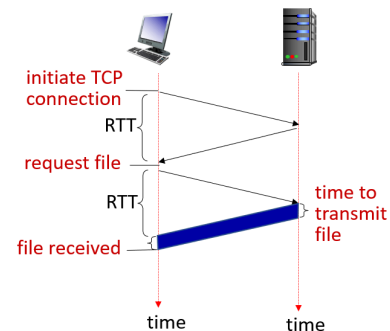
- Time-sensitive applications (VoIP, games) can run but must adapt (loss-tolerant design, adaptive coding).
- Internet provides best-effort delivery only.

Web & HTTP

- A **web page** is a collection of objects (files like HTML, JPEG, JS, CSS, video) identified by a URL (hostname + pathname, where the client (browser) requests content from a server (web server) in a client-server architecture with servers always on, fixed IPs, and capable of handling millions of requests.
- **HTTP**: client-server application-layer protocol.
 - Client: browser requests/receives/displays objects.
 - Server: web server sends objects in response. (e.g., Apache)
- **TCP port 80** for HTTP. TCP socket interface links HTTP to transport layer, Reliable data transfer
- **Stateless**: HTTP server stores no client state, Re-sends same object even if requested repeatedly.

HTTP Connection Types

1. **Non-persistent connections (HTTP/1.0)**: One TCP connection per object, requiring **2 RTTs (one to initiate TCP connection, one for HTTP request/response) + file** each time, causing OS overhead; browsers often open multiple parallel connections to fetch objects simultaneously. Avoids blocking and quickly frees server resources, but suffers from repeated handshake overhead and higher latency per object. (multiple rtts)
2. **Persistent connections (HTTP/1.1)**: A single TCP connection serves multiple objects, reducing RTTs and setup overhead; **1 RTT for connection + 1 RTT for each object**. Supports pipelining where clients send requests immediately for referenced objects, with the server keeping the connection open, Connection closed only after inactivity (timeout), often requiring as little as one RTT for all objects and cutting response time roughly in half. less overhead and better efficiency, but may cause blocking delays w large object and higher server load from long-lived connections.



RTT (definition): time for a small packet to travel from client to server and back

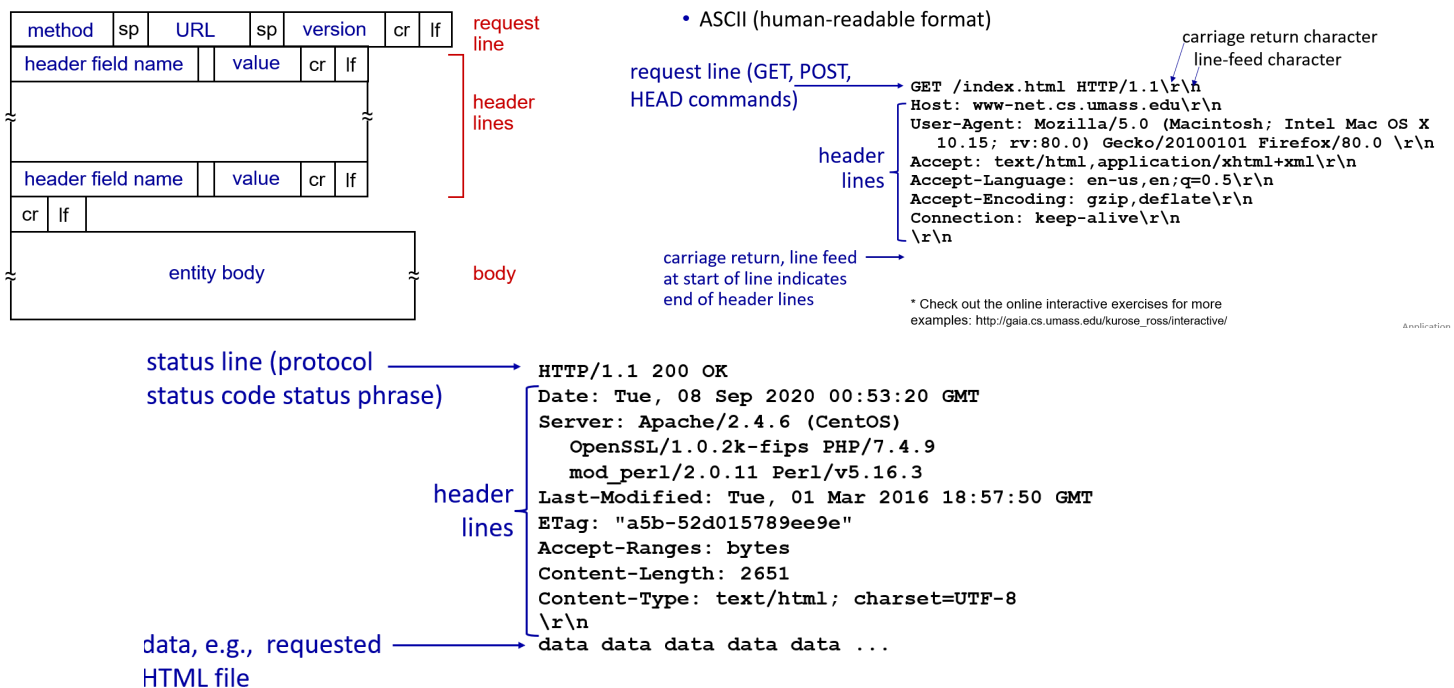
HTTP Methods:

- **POST**: Web page often includes form input; user input sent from client to server in entity body of HTTP POST request message.
- **GET**: Include user data in URL field of HTTP GET request message (following a ?), e.g., www.somesite.com/animalsearch?monkeys&banana.
- **HEAD**: Requests headers only that would be returned if specified URL were requested with an HTTP GET method.

- **PUT:** Uploads new file (object) to server; completely replaces file that exists at specified URL with content in entity body of POST HTTP request message.

HTTP Request & Response

- **HTTP Request Message:** An ASCII, human-readable message consisting of a request line (method + URL + version), header lines, a blank line, and an optional entity body (for POST).
- **HTTP Response Message:** Consists of a status line (protocol version, status code, phrase), header lines, and a data body (the object).
- Status codes: 200 OK, 301 Moved, 400 Bad Request, 404 Not Found, 505 Version Not Supported. appears in 1st line in server-to-client response message



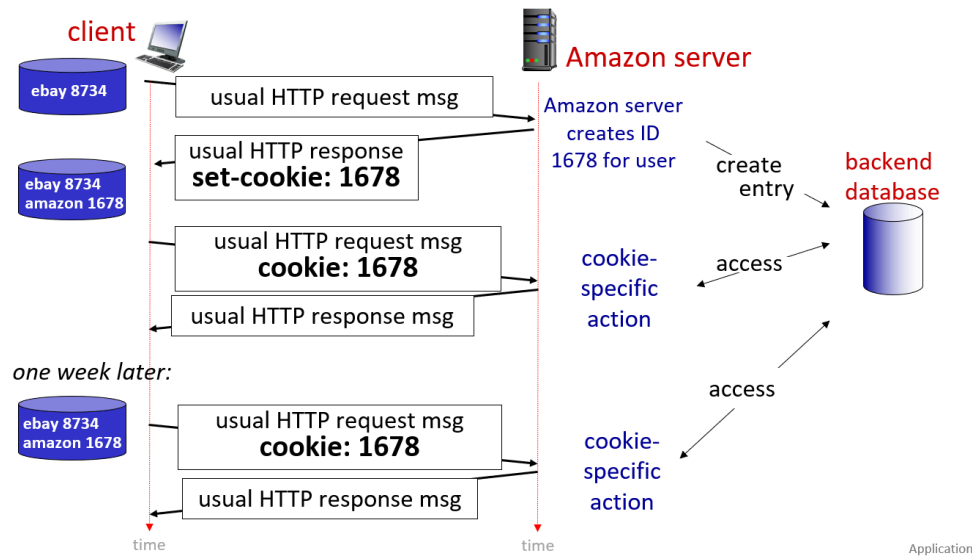
Example Non persistent HTTP (in 5 precise steps):

- Client initiates TCP connection to www.someSchool.edu on port 80, server accepts connection.
- Client sends HTTP request message for [/someDepartment/home.index](#).
- Server receives request, sends HTTP response with requested object (HTML file containing references to 10 JPEG images).
- Server closes TCP connection.
- Client receives HTML, parses and finds 10 JPEGs, repeats steps 1–4 for each JPEG.

Cookies & State

- HTTP is stateless but websites often need user identification. → cookies maintain state:
 1. Cookie in server response

2. Client sends cookie in future requests
 3. Stored in browser
 4. Backend database maps cookie to session
- Uses: authorization, shopping carts, recommendations, user session state
 - Cookies allow creating user session layer over stateless HTTP.
 - State maintenance: State can be maintained either at protocol endpoints (sender/receiver) or inside messages via cookies carrying state.
 - Privacy concerns: third-party cookies track across sites, GDPR regulates cookies. tracking, potential sale of user data. First party cookies only track on given site.

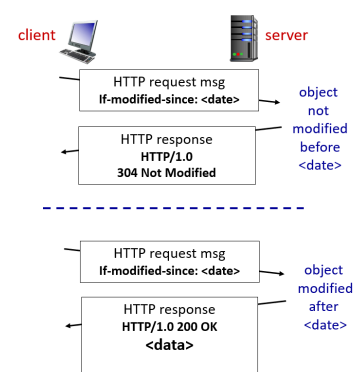


Example:

- Susan uses browser on laptop, visits e-commerce site for first time.
- Site creates unique ID (cookie) + entry in backend database.
- Subsequent HTTP requests contain cookie ID; site identifies Susan.

Web Caching (aka proxy servers)

- Browser points to local cache, cache serves objects if available, otherwise fetches from origin server, caches it, and return
- Web cache acts as both server for the requesting client and client to the origin server.
- Benefits: reduce response time, reduce traffic, improve performance, saves bandwidth
- Conditional GET (**If-modified-since** <date>) avoids unnecessary transfers. Ensures cached objects are up-to-date. Server responds with object only if modified; otherwise, reduces unnecessary data transfer. No transmission delay, no extra bandwidth
- Client:
 - includes If-Modified-Since: <date> in HTTP request.
- Server responds:



- HTTP/1.0 304 Not Modified if cached copy still valid → no object sent.
- HTTP/1.0 200 OK with data if object has been modified.
- **Content Distribution Networks (CDNs):** distributed caches to localize traffic (e.g., Akamai, Google, Netflix)

Caching Example – Without Cache

- Scenario:
 - Access link rate = 1.54 Mbps,
 - RTT to server = 2 sec,
 - Web object size = 100K bits,
 - Request rate = 15/sec,
 - Avg data rate to browsers = 1.50 Mbps.
- Performance:
 - Access link utilization = 0.97 (high),
 - LAN utilization = 0.0015 (very low),
 - End-to-end delay = Internet delay (2 sec) + access link delay (minutes) + LAN delay (microseconds).
- Problem: large queuing delays at high utilization.

$$\text{Utilization (U)} = \frac{\text{Average Data Rate (R)}}{\text{Link Capacity (C)}}$$

Option 1 – Buy Faster Link

- Same scenario but increase access link to 154 Mbps.
- Access link utilization becomes very low (~0.0097), delay in milliseconds, performance better but cost is expensive.

Option 2 – Install Local Cache

- Cache near client network, low cost solution.
- Requests can be served locally → reduces access link load, reduces delays.

Cache Performance with Hit Rate

- Suppose cache hit rate = 0.4:
 - 40% requests served locally (msec delay),
 - 60% requests go to origin.
- Access link rate to browsers = $0.6 \times 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$.
- Access link utilization = $0.9 / 1.54 \approx 0.58$ (moderate, low delay).
- Average end-to-end delay = $(0.6 \times 2.01 \text{ sec}) + (0.4 \times \text{msec}) \approx 1.2 \text{ sec}$.
- Cheaper and faster than buying expensive high-capacity link.

$$D_{\text{avg}} = (H \times D_{\text{cache}}) + (M \times D_{\text{origin}})$$

HTTP/2 & HTTP/3

- **HTTP/2:**
 - Goal: reduce delay in multi-object HTTP requests.

- HTTP/1.1 used pipelined GETs but FCFS caused Head-of-Line (HOL) blocking → small objects delayed behind large ones, loss recovery stalls transmission.
 - Retains same methods, status codes, headers.
 - Transmission order based on client-specified priority (not FCFS).
 - Server can push unrequested objects proactively.
 - Objects divided into frames, frames interleaved to mitigate HOL blocking.
- **HTTP/3:**
 - Limitation of HTTP/2: still over single TCP connection → loss recovery stalls all objects, no built-in security over TCP.
 - HTTP/3: runs over QUIC (Quick UDP Internet Connections). Deployed widely by Google (Chrome, YouTube app).
 - Built on UDP,
 - Includes reliability, congestion control, authentication, encryption, all in one protocol,
 - 1 RTT handshake for connection establishment,
 - 0 RTT for resumed sessions (using TLS session ticket),
 - Per-object error and congestion control,
 - Eliminates TCP head-of-line blocking.
- **QUIC:** combines reliability, congestion control, encryption, authentication, connection established in 1 RTT, supports 0-RTT handshakes

E-mail

- E-mail is an asynchronous communication medium: users send/read when convenient.
- Faster, cheaper, and easier to distribute than postal mail; supports attachments, HTML, hyperlinks, photos.
- **Three main components:**
 - **User Agents** (Outlook, Apple Mail, Gmail) → compose, read, reply, forward, save.
 - **Mail Servers** → store mailboxes, manage incoming/outgoing queues, retry delivery on failures.
 - **SMTP (Simple Mail Transfer Protocol)** → transfers mail between servers using TCP.

SMTP (RFC 5321)

- Restricts message body to **7-bit ASCII** → multimedia must be ASCII-encoded before sending and decoded on receipt.
- Uses **persistent TCP connections (port 25)** → multiple mails can be sent in one connection.
- **End of message indicated by CRLF.CRLF.**
- **Push protocol** (contrast: HTTP is pull).
- **HTTP:** one object per response; **SMTP:** multiple objects in a single multipart message.

```

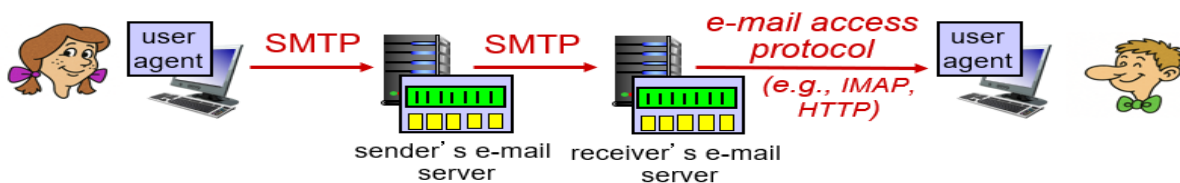
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr
C: MAIL FROM: <alice@crepes.fr>
S: 250 Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 Recipient ok
C: DATA
S: 354 Enter mail, end with "."
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 closing connection

```

- Email works through SMTP for sending and IMAP/POP for retrieving, while webmail (Gmail, Yahoo, etc.) uses HTTP on top of SMTP and IMAP/POP to provide a browser-based interface.

Operation (Alice → Bob):

1. Alice composes mail in her **User Agent (UA)** → sent to **Alice's mail server** and placed in **message queue**.
2. **SMTP client** on Alice's server opens TCP connection to **Bob's mail server**.
3. **Handshake**: HELO, MAIL FROM, RCPT TO, DATA, QUIT.
4. Message transferred reliably via TCP.
5. Bob's mail server stores mail in **Bob's mailbox**.
6. Bob retrieves it using his **UA** (via HTTP/IMAP/POP3).



Failure Handling:

- If recipient server is unavailable → sender's server retries periodically (e.g., every 30 min).
- If undeliverable after several days → message deleted + sender notified.

IMAP (Internet Mail Access Protocol) stores messages on the server and lets users retrieve, delete, and organize them into folders while keeping mail synchronized across devices.

POP3: Retrieves emails from the server and often downloads them locally, removing them from the server.

Domain Name System (DNS)

- Maps human-readable names ↔ IP addresses
- Distributed, hierarchical database, functions at the edge
- **No Centralized DNS:** Single point of failure, huge traffic volume, high latency due to centralized distance, maintenance difficulty, poor scalability (Comcast handles ~600B queries/day, Akamai ~2.2T queries/day).
- **Services:** Hostname-to-IP address translation, host aliasing (canonical vs. alias names), mail server aliasing (MX records), load distribution (replicated servers with many IPs per name).
- **Hierarchy:**
 - Root DNS servers → TLD DNS servers → Authoritative DNS servers → Local DNS servers.
- **Root Servers:** Official, contact-of-last-resort by name servers. 13 logical root servers worldwide; each replicated many times. Secured by DNSSEC (Domain Name System Security Extensions). **ICANN (Internet Corporation for Assigned Names and Numbers)** Manages the root DNS domain.
- **TLDs:** Responsible for generic TLDs (.com, .org, .net, .edu, .aero, .jobs, .museums) and country TLDs (.cn, .uk, .fr, .ca, .jp). Examples: Network Solutions (authoritative registry for .com, .net), Educause (.edu).
- **Authoritative:** Organization's own DNS servers providing hostname-to-IP mappings. Maintained by organization or service provider.
- **Local DNS Servers:** Each host sends its DNS query to a local DNS server (usually provided by the ISP). The local server may:
 - Return an answer from its cache (may be outdated).
 - Or forward the query into the DNS hierarchy for resolution.
 - Local servers do not belong strictly to the DNS hierarchy.
 - Found using OS tools (e.g., ipconfig /all or scutil --dns).
- **Queries:**
 - **Iterative:** server refers to next server, client iteratively queries down hierarchy.
 - **Recursive:** server resolves fully for client, heavier load on higher-level servers.
- **Caching**
 - Once a name server learns a mapping, it caches it for future queries.
 - Caching improves response time but entries expire after a TTL (Time-To-Live).
 - Cached entries can become out-of-date if a host changes its IP address before TTL expiry.
 - Thus, DNS offers best-effort name-to-address translation.

DNS Records

Type	Name	Value
NS	domain	authoritative server

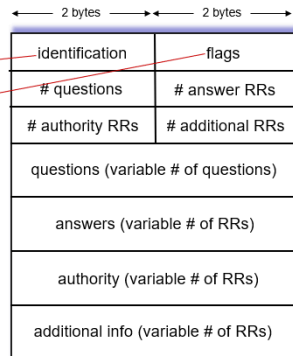
A	hostname	IP address
CNAME	alias	canonical hostname
MX	domain	SMTP mail server

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

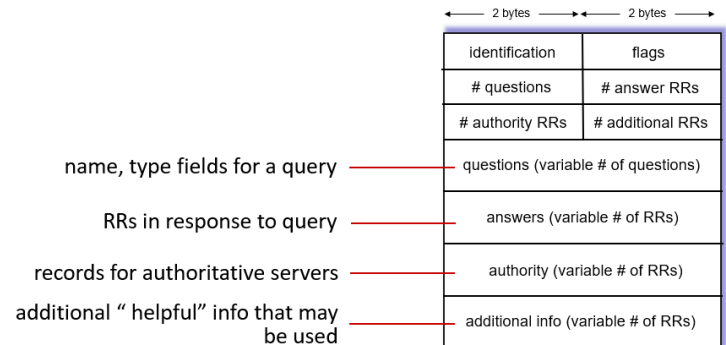
message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:



Getting Info into DNS

- New startup “Network Utopia” registers domain **networkuptopia.com** at DNS registrar (e.g., Network Solutions).
- Provide names and IP addresses of authoritative name servers (primary and secondary).
- Registrar inserts **NS (Name Server)** and **A (Address)** resource records into **.com** TLD (Top-Level Domain) server: (**networkuptopia.com**, **dns1.networkuptopia.com**, **NS**), (**dns1.networkuptopia.com**, **212.212.212.1**, **A**).
- Create local authoritative server with IP **212.212.212.1**.
- Create **A record** for **www.networkuptopia.com**.
- Create **MX (Mail Exchange) record** for **networkuptopia.com**.

DNS Security

DDoS attacks

- Attackers may bombard root or TLD servers with traffic.
- Mitigations include traffic filtering and reliance on local DNS server caches (which cache TLD server IPs and can reduce root server load).
- Attacks on TLD servers can be particularly dangerous.

Spoofing attacks

- Attackers can intercept DNS queries and return bogus replies.
- A common result is DNS cache poisoning, where cached entries are replaced with false mappings.

- Secure extensions (DNSSEC) provide authentication services and message integrity to protect against such attacks.

Chapter 3: Transport Layer

Transport Services and Protocols: They provide **logical communication** between application processes running on different hosts.

Household Analogy: The **hosts** are houses, the **processes** are kids, **app messages** are letters in envelopes, the **transport protocol** is Ann and Bill who demux to in-house siblings, and the **network-layer protocol** is the postal service.

Transport vs. Network Layer

- **Network Layer:** Provides communication between **hosts**.
- **Transport Layer:** Provides communication between **processes**, relying on and enhancing network layer services.
- **Sender Actions:** The application message is passed, the transport layer determines segment header field values, including sender/receiver port and packet sequence, creates a **segment**, and passes the segment to **IP** (network layer).
- **Receiver Actions:** The receiver gets the segment from IP, checks header values, extracts the application-layer message, and **demultiplexes** the message up to the application via a **socket**.

Internet Transport Protocols Summary

- **TCP (Transmission Control Protocol):** Provides **reliable, in-order delivery, congestion control, flow control, and connection setup**.
- **UDP (User Datagram Protocol):** Provides **unreliable, unordered delivery**, and is a **no-frills** extension of "best-effort" IP services.
- **Services Not Available:** The Internet transport protocols do not offer services like **delay guarantees** or **bandwidth guarantees**.

Multiplexing and Demultiplexing

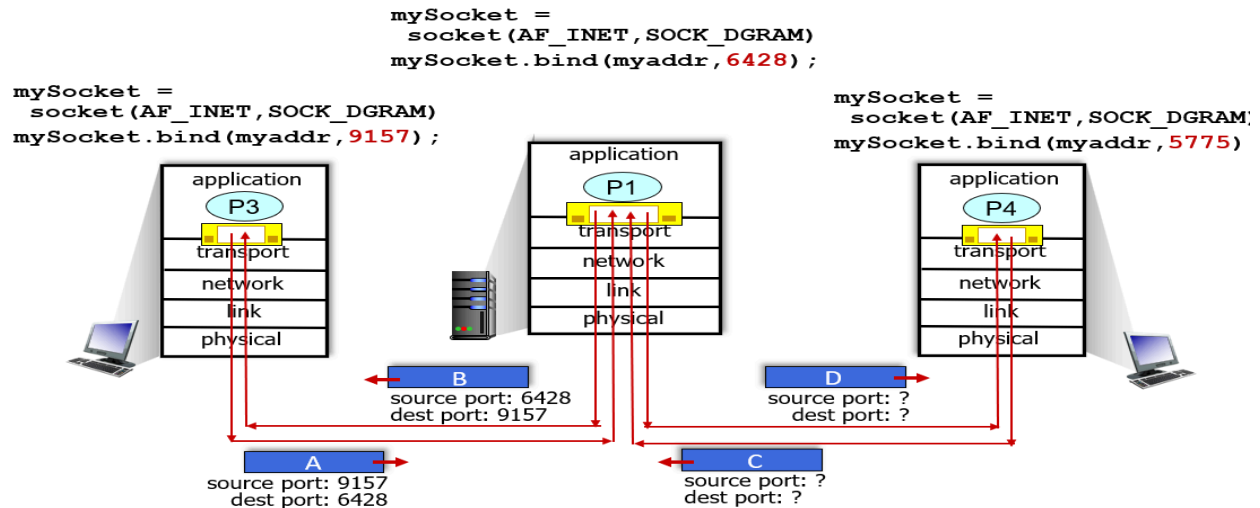
Definitions

- **Multiplexing (Sender):** The process of handling data from **multiple sockets** and adding a **transport header** (later used for demultiplexing).
- **Demultiplexing (Receiver):** The process of using **header info** to deliver received segments to the **correct socket**.
- **Mechanism:** A host receives **IP datagrams**, each containing a source IP address, a destination IP address, and carrying a transport-layer **segment**, which contains a source and destination 16 bit port number. The host uses the **IP addresses** and **port numbers** to direct the segment to the appropriate **socket**.

Connectionless Demultiplexing (UDP)

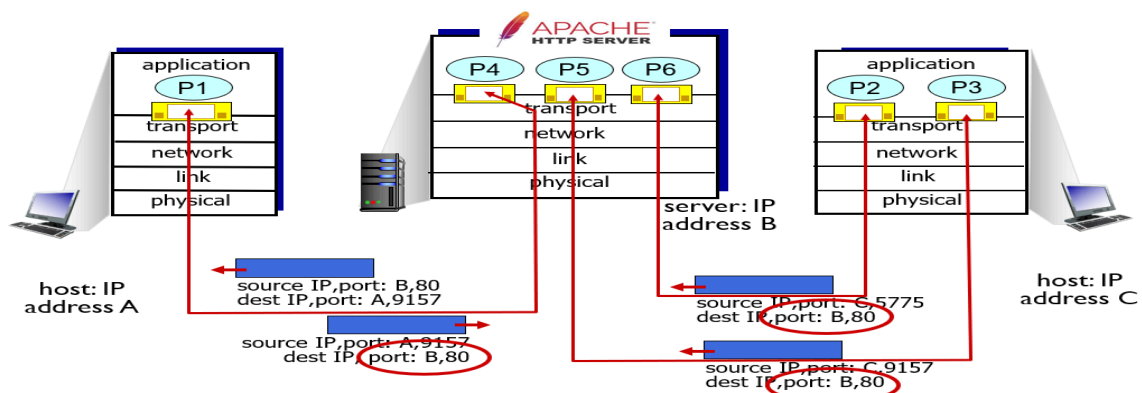
- **Socket Identification:** A UDP socket is fully identified by a two-tuple: destination IP address and destination port number.

- **Logic:** The receiving host checks only the **destination port #** in the segment and directs the UDP segment to the socket with that port #.
- **Example:** IP/UDP datagrams with the **same destination port #**, but **different source IP addresses** and/or **source port numbers**, will be directed to the **same socket** at the receiving host.



Connection-Oriented Demultiplexing (TCP)

- **Socket Identification:** A TCP socket is uniquely identified by a **4-tuple**: **source IP address**, **source port number**, **dest IP address**, and **dest port number**.
- **Mechanism:** The receiver uses **all four values** (the 4-tuple) to direct the segment to the appropriate socket.
- **Distinction from UDP:** Two arriving TCP segments with **different source IP addresses** or **source port numbers** will be directed to **two different sockets** (except for the initial connection-establishment segment).
- **Server Example:** A server may support many simultaneous TCP sockets, with each socket identified by a unique 4-tuple associated with a different connecting client.



Three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

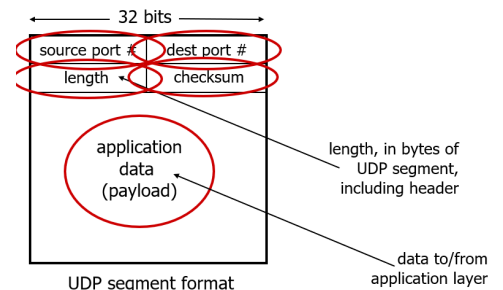
UDP: User Datagram Protocol

Characteristics and Use

- **Description:** A "no frills," "bare bones" Internet transport protocol.
- **Service:** Provides "**best effort**" service, meaning UDP segments may be **lost** or delivered **out-of-order** to the application.
- **Minimal Functionality:** UDP adds little to the IP service model, providing only multiplexing/demultiplexing (to pass data to the correct application process) and some light error checking (checksum).
- **Efficiency:** There is **no connection establishment** (avoiding RTT delay), it is **simple** (no connection state at sender/receiver), it has a **small header size**, and **no congestion control** (it can blast away as fast as desired and function in the face of congestion).
- **Use Cases:** Streaming multimedia apps (which are loss tolerant, rate sensitive), **DNS**, **SNMP**, and **HTTP/3**. If reliability is needed, it must be added at the **application layer**.
- **UDP Sender Action:** The sender receives the application message, determines the header fields (including port numbers and checksum), creates the UDP segment, and immediately passes it to IP.
- **UDP Receiver Action:** The receiver gets the segment from IP, checks the checksum, extracts the application message, and uses the destination port number to demultiplex the message up to the correct application process via its socket.

UDP Segment Format

- **Fields:** The header includes **source port #**, **dest port #**, **length** (in bytes of the UDP segment, including header), and **checksum**.



UDP Checksum

- **Goal:** To detect **errors** (i.e., flipped bits) in the transmitted segment.
- **Sender Process:** Treats segment contents as **16-bit integers**, computes the **one's complement sum** (the checksum) of the content, and puts the value into the checksum field.
- **Receiver Process:** Computes the checksum of the received segment.
 - **Error Detected:** If the computed checksum is **not equal** to the field value.
 - **Weak Protection:** If the values are equal, no error is detected, but errors may nonetheless be present, indicating it is **weak protection**.
- **One's Complement Sum Example:** When adding numbers, a **carryout** from the most significant bit must be **added to the result** (known as wraparound).
- The **UDP checksum** can fail to detect errors if **multiple bit errors** occur that **cancel each other out** in the final **1s complement sum**.
- **Error Handling:** UDP only provides **error detection**, not recovery. Some implementations **discard the damaged segment**; others pass it to the application **with a warning**.

example: add two 16-bit integers

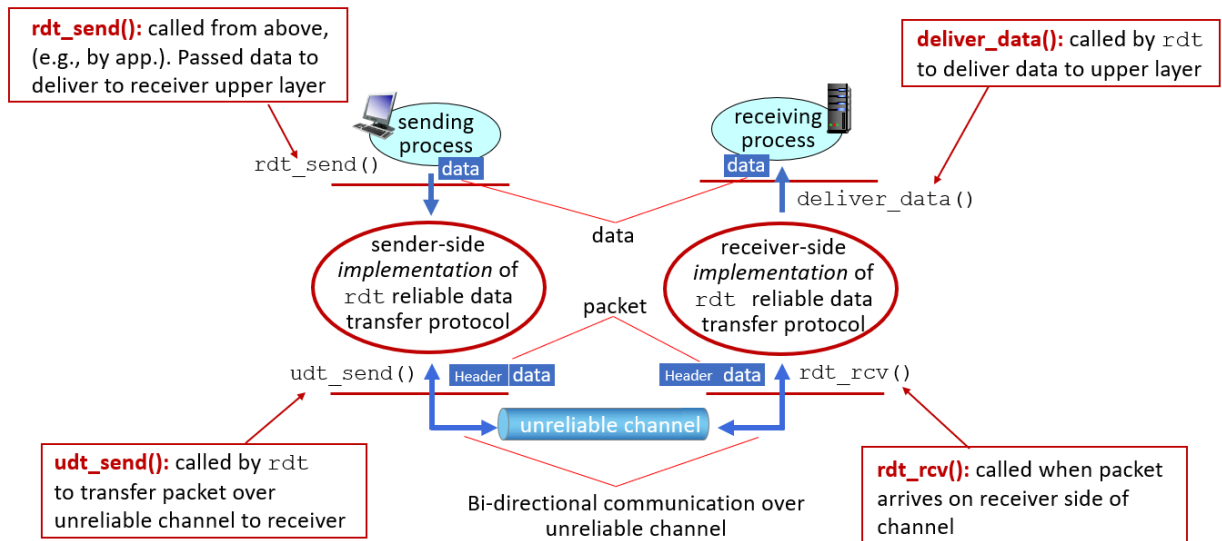
	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Principles of Reliable Data Transfer (rdt) - Conceptual Foundation for TCP

Core Concepts

- **Goal:** To ensure that data sent from a sender reaches the receiver **correctly, completely, and in the original order**, even when the underlying communication channel is unreliable (i.e., may lose or corrupt packets).
- **Reliable Data Transfer (rdt) Protocol:** Mechanism (e.g., TCP) ensuring reliable delivery of data over an unreliable point-to-point channel.
- **Reliable Channel:** Upper layer service abstraction guaranteeing no loss, corruption, and in-order delivery of data.
- **Unreliable Channel:** Underlying channel (e.g., IP) that may lose or corrupt packets but does not reorder them. Bidirec
- **Unidirectional Data Transfer:** Simplified model where application data flows from sender → receiver; control packets may flow both ways. Data transfer is unidirectional but communication channel is bidirectional.
- **Control Packets:** Non-data packets (e.g., ACKs) used to manage reliable delivery.
- **Abstraction:** RDT provides a reliable service over an unreliable channel.
- **Protocol Interfaces:**
 - `rdt_send()`: Called by the application, passes data to deliver.
 - `udt_send()`: Called by rdt to transfer a packet over the unreliable channel.
 - `rdt_rcv()`: Called when a packet arrives on the receiver side.
 - `deliver_data()`: Called by rdt to deliver data to the upper layer.
- **Protocol Development:** Sender and receiver sides are developed incrementally using **Finite-State Machines (FSMs)**, assuming that the underlying channel **will not reorder packets**.



Version / Name	Description
rdt1.0 – Perfectly Reliable Channel	This version assumes a completely reliable channel with no bit errors or packet loss. The protocol is trivial, consisting of a single-state FSM for both sender and receiver. The sender simply accepts data through <code>rdt_send(data)</code> , creates a packet using <code>make_pkt(data)</code> , and sends it via <code>udt_send(packet)</code> . Since the channel is flawless, no acknowledgments (ACKs) or negative acknowledgments (NAKs) are required, as nothing can go wrong. Overall, rdt1.0 represents a basic stop-and-wait model operating under ideal conditions.
rdt2.0 – Channel with Bit Errors	rdt2.0 assumes the communication channel can corrupt bits but delivers packets in order. It is an ARQ (Automatic Repeat reQuest) protocol based on retransmissions and uses error detection and feedback for reliability. The receiver detects errors using a checksum and sends either an ACK (packet OK) or NAK (packet error). The sender retransmits upon receiving a NAK, and the protocol operates in a stop-and-wait fashion where the sender waits for an ACK before sending new data. However, a major flaw exists—if the ACK or NAK itself is corrupted, the sender cannot determine the receiver's state, leading to possible duplicate transmissions.

rdt2.1 – Handling Corrupted Feedback (Garbled ACK/NAK)	rdt2.1 addresses the fatal flaw in rdt2.0 caused by corrupted ACKs or NAKs that made the sender uncertain about the receiver's state. To fix this, a sequence number field is added to packets so the receiver can identify whether a packet is new or a retransmission. A 1-bit sequence number (0 or 1) is sufficient for this stop-and-wait protocol. As a result, both sender and receiver FSMs now have double the states to remember which sequence number they are handling. The receiver discards duplicate packets and keeps track of the expected sequence number, thereby ensuring correct data delivery even if feedback packets are corrupted.
rdt2.2 – NAK-free Protocol	rdt2.2 provides the same reliability as rdt2.1 but eliminates the use of NAKs altogether. Instead of sending NAKs when an error occurs, the receiver sends a duplicate ACK for the last correctly received packet, with each ACK including the sequence number of the acknowledged packet. The sender treats a duplicate ACK as an implicit NAK and retransmits the current packet. By relying only on ACKs, rdt2.2 simplifies feedback handling and enhances robustness against corrupted control messages while maintaining reliable data transfer.
rdt3.0 – Lossy Channel with Bit Errors	rdt3.0 extends reliability to channels that can both corrupt and lose packets, including data or ACKs. To handle potential loss, the sender uses a countdown timer that starts when a packet is sent and stops upon receiving a valid ACK. If no ACK is received before the timeout, the sender assumes packet loss and retransmits the last packet. This version combines checksums for error detection, sequence numbers for duplicate handling, timers for loss recovery, and ACKs for confirmation. It is also known as the alternating-bit protocol because sequence numbers alternate between 0 and 1. Although it achieves full reliability, its stop-and-wait design results in low sender utilization, limiting performance efficiency.

SEE FSMS FROM SLIDES

rdt3.0 Performance (Stop-and-Wait):

- **Transmission Delay (D_{trans}):** $D_{\text{trans}} = L/R$ (e.g., 8000 bits/ 10^9 bits/sec = 8μsecs).
- **Utilization (U_{sender}):** The fraction of time the sender is busy sending.

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

. The protocol limits the performance of the underlying channel.

Pipelined Protocols (Go-Back-N and Selective Repeat)

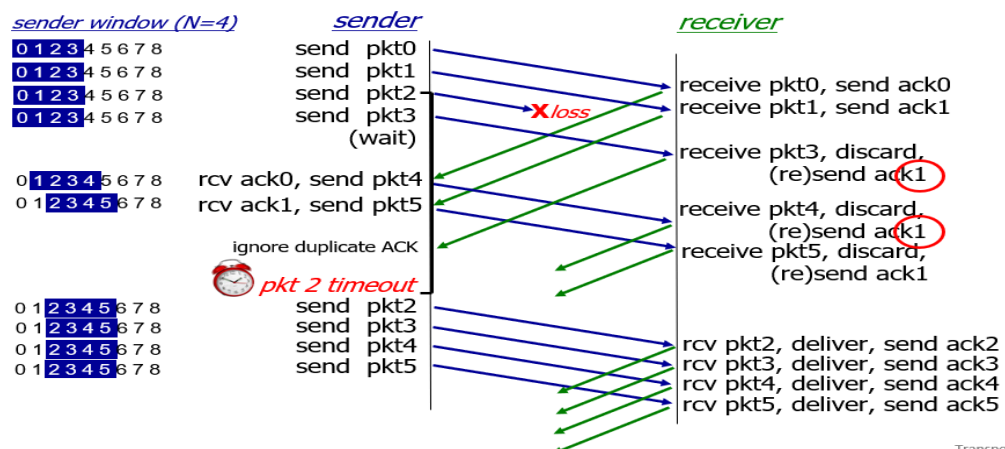
Pipelining Principle

- **Performance Issue:** rdt3.0 is a stop-and-wait protocol, which results in dismal sender utilization (Userender) and low effective throughput in high-speed networks.
- **Example:** On a 1 **Gbps** link with a 30 **msec RTT**, the sender utilization is only $U_{\text{sender}} = (L/R) / (RTT + L/R) \approx 0.00027$ (0.027% of the time).
- **Solution: Pipelining:** The sender is allowed to send **multiple packets** without waiting for acknowledgments, essentially **filling the channel pipeline** with in-transit packets.
- **Benefit:** Pipelining significantly increases sender utilization and effective throughput, solving the stop-and-wait performance problem.
- **Requirements:** The range of sequence numbers must be increased, and buffering is needed at the sender and/or receiver.

Go-Back-N (GBN)

- **Sender Window:** The sender maintains a "window" of up to N consecutive transmitted but unACKed packets.
- **ACK Type:** Uses **cumulative ACK**: $ACK(n)$ An **ACK** for a packet with sequence number n is interpreted as indicating that **all packets** with sequence numbers **up to and including n** have been correctly received. on receiving ACK(n): move window forward to begin at n+1
- **Retransmission:** A timeout for packet n causes the sender to **retransmit packet n and all higher seq # packets in the window**.
- **Receiver:**
 - **ACK Logic:** Always sends an **ACK for the correctly-received packet so far**, with the **highest in-order seq #**. may generate duplicate ACKs, need only remember rcv_base (next expected packet)
 - **Out-of-Order Packet:** The receiver **discards** the packet (or buffers, depending on implementation) and re-ACKs the packet with the highest in-order seq #.

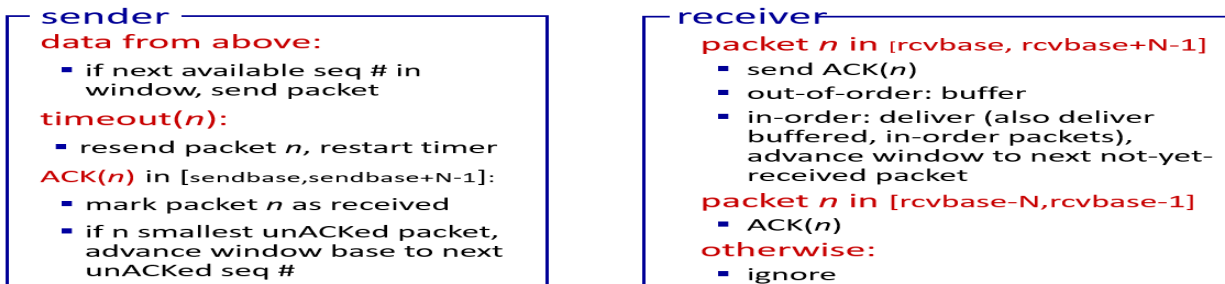
Go-Back-N in action



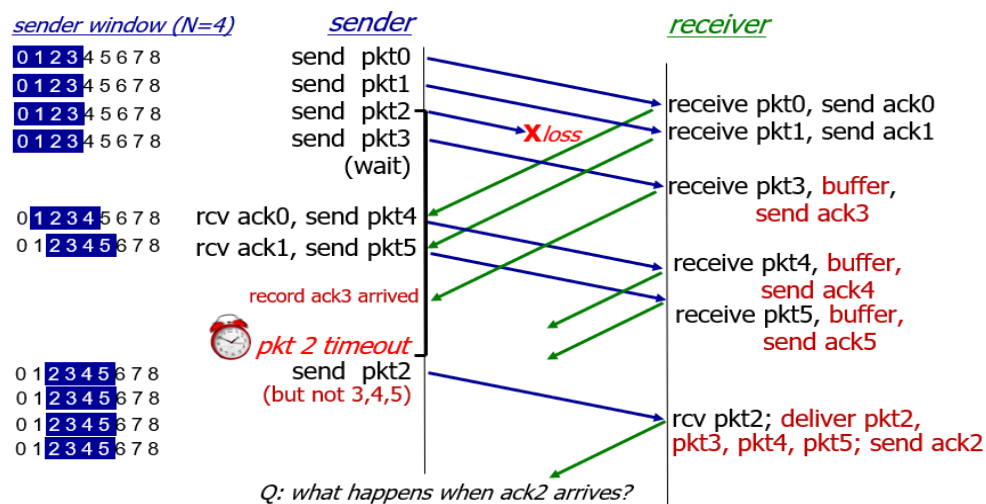
Selective Repeat (SR)

- The sender retransmits only those packets that it suspects were lost or corrupted, avoiding the "go-back-N" behavior.
- ACK Type:** The receiver **individually ACKs all correctly received packets**.
- Receiver Action:** The receiver **buffers** out-of-order packets for in-order delivery to the upper layer.
- Sender Timer:** Maintains a **timer for each unACKed packet** (conceptually).
- Retransmission:** A timeout for packet n causes the sender to **retransmit only the single unACKed packet n** . Each individual packet must have its own logical timer because only that specific packet is retransmitted upon timeout.
- Window:** A sliding window controls up to N consecutive sequence numbers, limiting the number of unacknowledged ('in-flight') packets in pipelined transmission
- Dilemma:** To avoid a retransmitted packet from being mistaken for a new one, the **window size (N) must be less than or equal to half of the sequence number space size**.

Selective repeat: sender and receiver



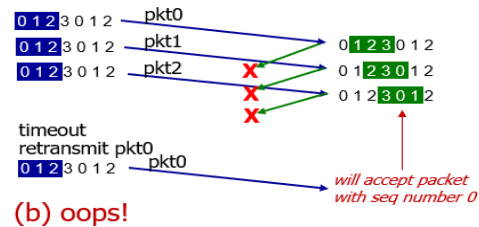
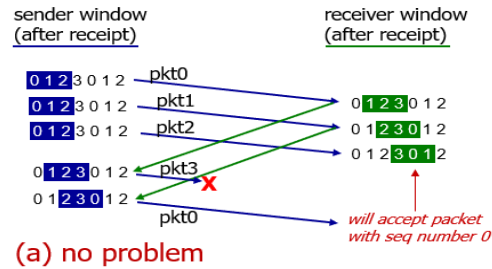
Selective Repeat in action



Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

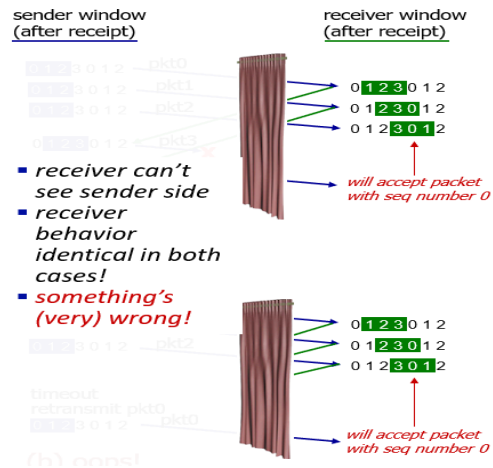


Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

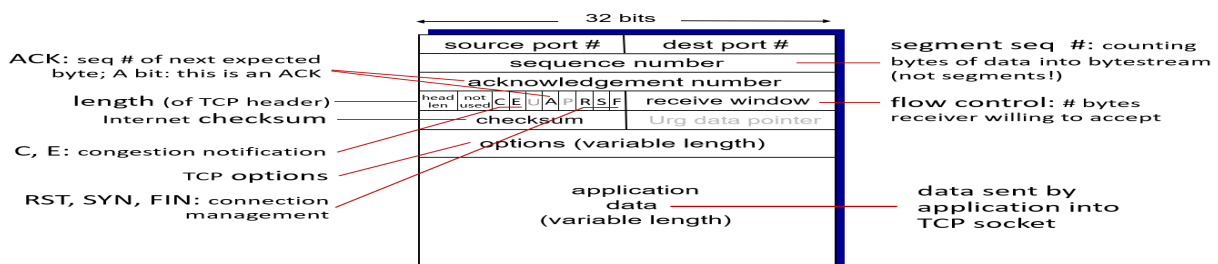
Q: what relationship is needed between sequence # size and window size to avoid problem in scenario (b)?



TCP: Connection-Oriented Transport

TCP (Transmission Control Protocol) provides a reliable, connection-oriented, point-to-point communication between a sender and receiver. It ensures in-order delivery of a continuous byte stream, supports full-duplex (bi-directional) data transfer, and uses handshaking to establish connections. Flow control prevents the sender from overwhelming the receiver, while pipelining and congestion control regulate the number of in-flight segments using a sliding window. TCP also defines the Maximum Segment Size (MSS) to optimize data transmission.

TCP Segment Structure



- **Data/Flow Control Fields:** Includes **source port #**, **dest port #**, **application data**, and the **receive window (rwnd)**, which is the # bytes the receiver is willing to accept for flow control.
- **Reliability Fields:**
 - **Sequence number:** The byte stream **number of the first byte** in the segment's data.
 - **Acknowledgement number (ACKnum):** The sequence number of the **next expected byte**.
 - **ACK bit (A):** Indicates this is an ACK segment.
 - **Checksum:** The Internet checksum.
- **Control/Setup Fields:** Includes the **header length**, **options**, and flags like **RST**, **SYN**, and **FIN** for connection management.

Sequence Numbers and ACKs

- **Sequence Numbers:** These count **bytes of data** into the byte stream, not segments.
- **Acknowledgements:** Are **cumulative** and specify the seq # of the **next byte expected** from the other side.
- **Out-of-Order Handling:** The TCP specification does not mandate how the receiver handles out-of-order segments; it is **up to the implementor**.

The **maximum TCP segment size (MSS) $\approx$ 64 KB (65,535 bytes)**, because:

- TCP header's **window size field is 16 bits** $\rightarrow$ max value = 65,535 bytes.
- MSS $\approx$ 64 KB is the largest segment size possible without IP fragmentation.

TCP Round-Trip Time (RTT) & Timeout:

- **Timeout Setting:** Must be longer than RTT to avoid premature retransmission; too long slows loss detection.
- **Estimating RTT:** Use **SampleRTT** (time from segment sent until ACK received), ignoring retransmissions.
- **Smoothed RTT:** Average several recent **SampleRTTs** for a stable, adaptive timeout estimate.

TCP RTT & Timeout Estimation:

- **Estimated RTT:** Calculated using an **exponential weighted moving average (EWMA)**:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

where $\alpha \approx 0.125$. This smooths out fluctuations by giving more weight to recent samples.

- **Deviation (DevRTT):** Measures variability of RTT:

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

typically $\beta \approx 0.25$.

- **Timeout Interval:** Adds a safety margin to handle RTT variations:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

- Short timeout → premature retransmissions
- Long timeout → slow reaction to segment loss

This method allows TCP to adapt dynamically to network conditions, smoothing RTT fluctuations while maintaining reliable timeout behavior.

Notes:

- SampleRTT = time between segment transmission and ACK reception.
- Retransmitted segments are not used in RTT estimation.
- EWMA smoothing prevents wild fluctuations in timeout intervals.

TCP Sender (Simplified):

1. **Data from Application:**
 - Creates a segment with a sequence number (first byte of the segment).
 - Starts a timer for the oldest unACKed segment (**TimeoutInterval**).
2. **Timeout Event:**
 - Retransmits the segment that caused the timeout.
 - Restarts the timer.
3. **ACK Received:**
 - Updates the highest byte acknowledged.
 - If unACKed segments remain, restarts the timer.

TCP Receiver ACK Generation:

1. **In-Order Segment, No Pending ACK:** Delays ACK up to 500 ms (delayed ACK) to wait for the next segment; if none arrives, sends ACK.
2. **In-Order Segment, One Pending ACK:** Sends a single cumulative ACK immediately, acknowledging both segments.
3. **Out-of-Order Segment (Gap Detected):** Sends an immediate duplicate ACK indicating the last correctly received in-order byte and the next expected sequence number.

Retransmission Scenarios

Loss + Timeout: A segment is lost; when its timer expires, the sender retransmits it.

Premature Timeout: A timeout occurs just before an ACK arrives, triggering an unnecessary retransmission. The receiver may receive a duplicate, but sequence numbers ensure correct ordering.

TCP Fast Retransmit: If the sender receives **three duplicate ACKs** for the same data, it immediately **retransmits the unACKed segment with the smallest sequence number** without waiting for a timeout, because the duplicates indicate a likely lost segment.

TCP Flow Control

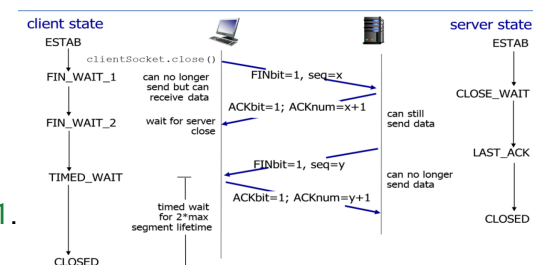
- **Purpose:** Prevents the **sender** from overflowing the **receiver's buffer** when incoming data arrives faster than the application can process it.
- **Mechanism:** The receiver advertises available buffer space in the `rwnd` field of the TCP header. The sender limits the amount of unACKed ("in-flight") data to the advertised `rwnd`, ensuring the receive buffer never overflows. Receive buffer sizes can be set via socket options or auto-adjusted by the OS (typical default ~4096 bytes).
- **Ensures:**
$$LastByteSent - LastByteACKed \leq rwnd$$
- `rwnd` dynamically changes based on the receiver's current buffer occupancy.
- Guarantees reliable, flow-balanced delivery even if receiver reads data slowly.

TCP Connection Management

- Before data exchange, TCP uses a 3-way handshake to ensure both sides are ready and agree on initial sequence numbers and other connection parameters.
- A 2-way handshake fails in networks with variable delays, retransmissions, or message reordering, as it can cause duplicate or false/half open connections.

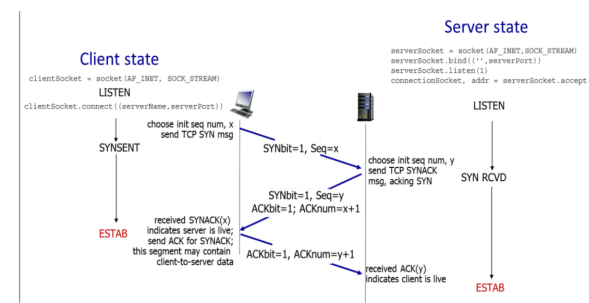
A. Connection Establishment (3-Way Handshake)

1. **Client → Server:**
Sends **SYN** (synchronize) segment with **SYN=1, Seq=x**.
2. **Server → Client:**
Sends **SYN-ACK** with **SYN=1, ACK=1, Seq=y, AckNum=x+1**.
3. **Client → Server:**
Sends **ACK** with **ACK=1, AckNum=y+1**.
Connection established — data transfer may begin.



B. Connection Termination (4-Step Process)

1. **Client → Server:** FIN segment (**FIN=1, Seq=x**).
 2. **Server → Client:** ACK for client's FIN (**AckNum=x+1**).
 3. **Server → Client:** FIN segment (**FIN=1, Seq=y**).
 4. **Client → Server:** ACK for server's FIN (**AckNum=y+1**).
- simultaneous FIN exchanges can be handled



TIME_WAIT: The client waits $2 \times \text{MSL}$ (**Maximum Segment Lifetime**) before closing to ensure all delayed segments and ACKs have been processed — prevents confusion from old connections.

TCP Congestion Control: AIMD (Additive Increase, Multiplicative Decrease)

- **Approach:** Senders **increase** the sending rate until **packet loss (congestion)** occurs, and then **decrease** the sending rate on the loss event.
- **Additive Increase:** Increase the sending rate by **1 Maximum Segment Size (MSS)** every **RTT (Round-Trip Time)** until loss is detected.
- **TCP Multiplicative Decrease:** On loss detection, the sender reduces its sending rate:
 - **TCP Reno:** Halves the congestion window (cwnd) when loss is indicated by triple duplicate ACKs.
 - **TCP Tahoe:** Reduces cwnd to 1 MSS when loss is detected by timeout.
- **Behavior:** The result is a **sawtooth behavior** where the sender is continually **probing for bandwidth**.
- **Why AIMD?** It is a distributed, asynchronous algorithm shown to:
 - **Optimize congested flow rates network wide.**
 - Have **desirable stability properties.**

TCP Congestion Control: Details

- The **congestion window (cwnd)** limits the amount of data a TCP sender can have "in-flight" (sent but not yet acknowledged).
- **TCP Sending Behavior:** Roughly: send **cwnd** bytes, wait RTT for ACKs, then send more bytes.
- **TCP Sender Limit:** The sender limits transmission based on the window: **$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$** .
- **TCP Rate Approximation:** **$\text{TCP rate} \approx \text{RTT}/\text{cwnd}$ bytes/sec.**
- **Dynamic Adjustment:** **cwnd** is dynamically adjusted in response to observed **network congestion** (implementing TCP congestion control).
- **Sender Sequence Number Space:** The window spans three regions:
 - **Last byte ACKed:** The boundary of acknowledged data.
 - **Sent, but not-yet ACKed ("in-flight"):** The data currently outstanding, bounded by **cwnd**.
 - **Available but not used:** The space after the last byte sent.

TCP Congestion Control Phases (RENO)

1. Slow Start

- When a TCP connection begins, **cwnd (congestion window)** starts at **1 MSS**, giving an initial sending rate of **MSS/RTT**.
- TCP increases **cwnd by 1 MSS for each ACK received**, causing **cwnd to double every RTT** (exponential growth).

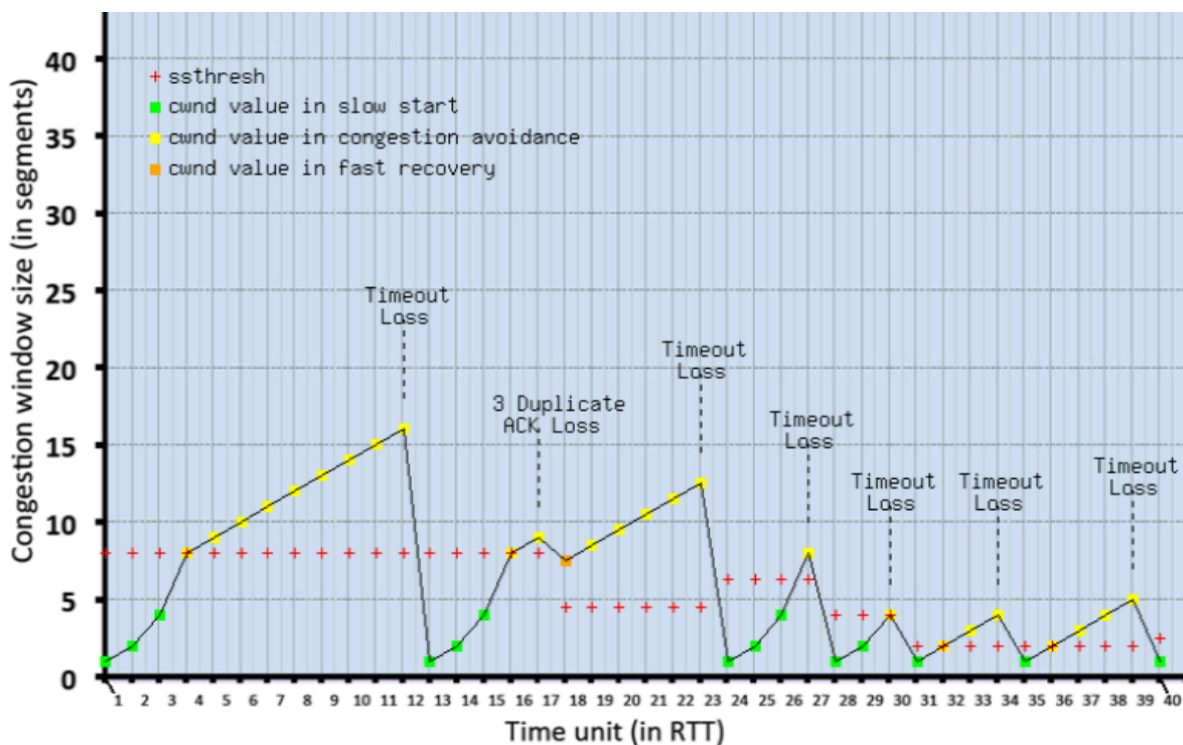
- **Slow start ends** when:
 1. A **timeout (loss event)** occurs → cwnd reset to **1 MSS**, **ssthresh = cwnd/2**, restart slow start.
 2. **cwnd ≥ ssthresh** → transition to **Congestion Avoidance**.
 3. **Three duplicate ACKs** are detected → **fast retransmit**, enter **Fast Recovery**.

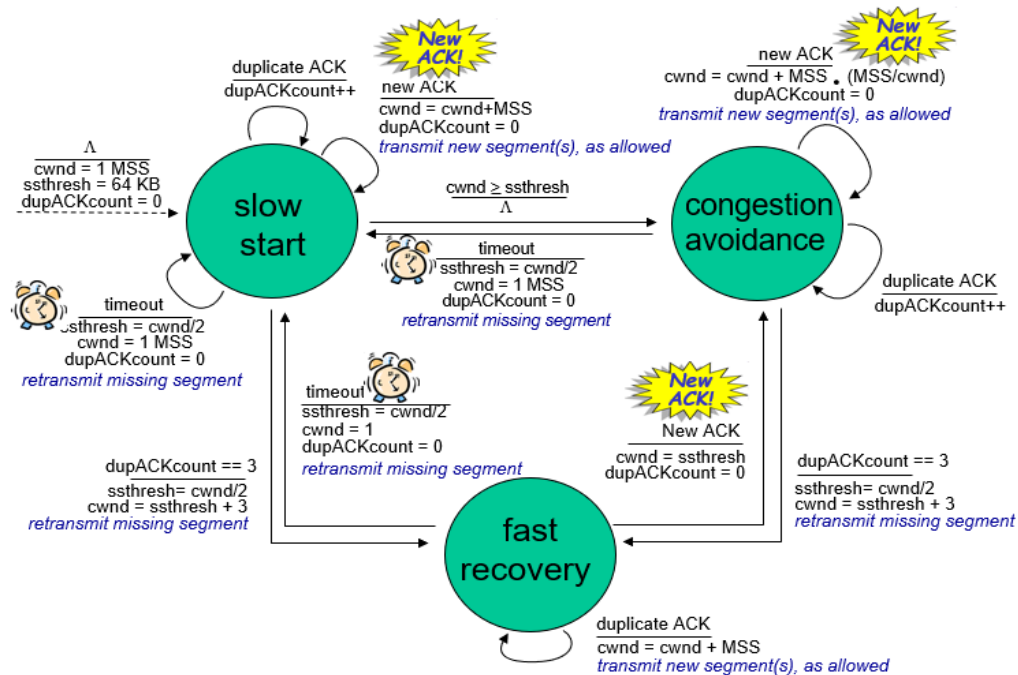
2. Congestion Avoidance

- Starts when **cwnd ≈ ssthresh** (about half of previous cwnd at congestion).
- To avoid congestion, TCP **increases cwnd linearly** by **1 MSS per RTT**.
 - Practically: cwnd increases by a small fraction ($MSS/cwnd$) for each ACK.
- **On loss:**
 - **Timeout:** cwnd = 1 MSS, **ssthresh = cwnd/2**, restart slow start.
 - **Triple duplicate ACKs:** cwnd halved (+3 MSS for received ACKs), **ssthresh = cwnd/2**, enter **Fast Recovery**.

3. Fast Recovery

- On entering fast recovery, **cwnd increases by 1 MSS for every duplicate ACK** received.
- When the **ACK for the missing segment** arrives, TCP **deflates cwnd** and returns to **congestion avoidance**.
- If a **timeout** occurs, cwnd = 1 MSS and **ssthresh = cwnd/2**, returning to **slow start**.
- **TCP Reno** includes fast recovery, while **TCP Tahoe** does not—it always resets cwnd to 1 MSS and re-enters slow start after any loss.





Alternative Congestion Control: TCP CUBIC

- Designed to **improve bandwidth utilization** over classic AIMD (Additive Increase Multiplicative Decrease) TCP.
- Uses **W_{max}**: the window size at which congestion (loss) was last detected.
- After loss ($cnwnd$ set to half), the window initially ramps quickly toward **W_{max}**, then approaches it **cautiously** as it nears **W_{max}**.
- Window growth is **cubic** with respect to the time difference from a tunable point **K** (when the window will reach **W_{max}**):
 - Far from **K** → larger increases (aggressive)
 - Near **K** → smaller increases (conservative)
- Default TCP in Linux; widely used on web servers until ~2024.

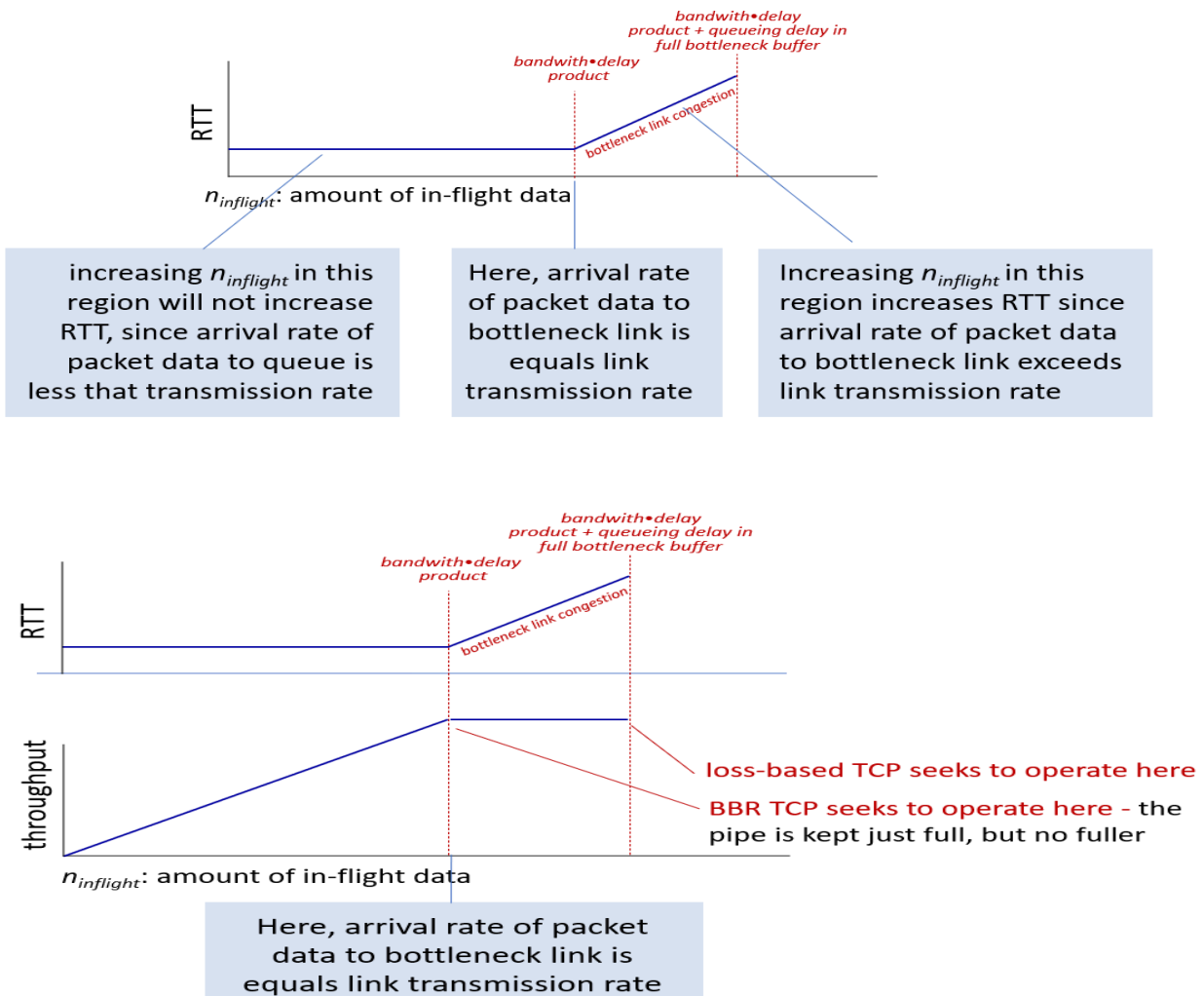
TCP and the Bottleneck Link

- Bottleneck Link**: The router's output link where a packet loss (congestion) occurs because the queue overflows.
- TCP (classic, CUBIC) increases the sending rate until packet loss occurs at this point.
- Insight on Congestion**:
 - Increasing the TCP sending rate **will not increase end-to-end throughput** at the congested bottleneck, as it is **already transmitting at its full rate, R**.
 - Increasing the TCP sending rate **will increase the measured RTT** due to **increased queueing delay** at the bottleneck link.

- **Goal:** The ultimate goal is to “keep the end-end pipe just full, but not fuller”. ensuring fair throughput among multiple flows (R/N per flow if N flows share capacity R).

Keeping the Pipe Just Full Enough:

- **Goal:** Keep the bottleneck link fully utilized but avoid excessive queuing and delay.
- **In-Flight Data ($n_{inflight}$):** Ideal amount $\approx$ connection's share of bottleneck bandwidth $\times$ min RTT (the bandwidth-delay product).
- **Behavior:**
 - If $n_{inflight} <$ bandwidth-delay product $\rightarrow$ link underutilized; RTT low, throughput limited.
 - If $n_{inflight} \approx$ bandwidth-delay product $\rightarrow$ link fully utilized; RTT stable; optimal throughput.
 - If $n_{inflight} >$ bandwidth-delay product $\rightarrow$ queue builds; RTT increases; potential packet loss.
- **TCP Strategies:**
 - Loss-based TCP operates near full queues (may cause higher RTT).
 - BBR TCP aims to keep the pipe just full without excess queuing, maintaining high throughput with low delay.



BBR Congestion Control: BBR (Bottleneck Bandwidth and RTT)

- A congestion control algorithm that directly **regulates ninfight** (amount of in-flight data).
- **Regulates in-flight data (n_inflight)** to maximize throughput without overfilling queues.
- **Long intervals:** Reduce **n_inflight** to drain the pipe and measure new **min_RTT**.
- **Short intervals:** It operates in phases:
 - **Acceleration:** Increase sending rate and **n_inflight** until throughput saturates the link.
 - **Cruising:** Maintain sending rate matching observed delivery rate via ACKs.
 - **Deceleration:** Reduce **n_inflight** to relieve queues and find lower **min_RTT**.

Explicit Congestion Notification (ECN): A network-assisted congestion control mechanism where routers mark two bits in the IP header to signal congestion. The destination echoes this by setting the ECE bit in the TCP ACK, notifying the sender of congestion before packet loss occurs. ECN thus involves both IP header marking and TCP header signaling to manage congestion proactively.

TCP Fairness

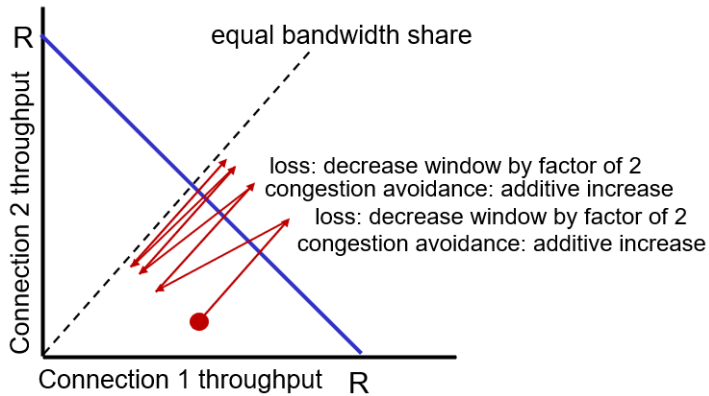
- **Fairness Goal:** If **K TCP sessions** share the same bottleneck link of bandwidth **R**, each should have an average rate of **R/K**.
- **Is TCP Fair?** Yes, under idealized assumptions (same RTT and a fixed number of sessions only in congestion avoidance). The **additive increase** part gives a slope of 1, and the **multiplicative decrease** decreases throughput proportionally, leading to equal sharing in the long run.

Fairness, UDP, and Parallel Connections

- **Fairness and UDP:** Multimedia applications often use **UDP** because they **do not want their rate throttled by congestion control**; they send audio/video at a constant rate and tolerate packet loss.
- **Fairness, parallel TCP connections:** An application can open multiple parallel TCP connections between two hosts (e.g., web browsers do this). If a link of rate **R** has 9 existing connections:
 - A new app asking for **1 TCP** gets rate **R/10**.
 - A new app asking for **11 TCPs** gets rate **R/2** (as it now holds 11 out of 21 connections, or $\approx R/2$).

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughput increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Chapter 4: Network Layer

Services and Protocols: The network layer's primary service is to **transport segment from sending to receiving host**.

- **Sender:** encapsulates segments into datagrams, passes to link layer.
- **Receiver:** delivers segments to transport layer protocol.

Network layer protocols are in **every Internet device: hosts and routers**.

- **Routers** must:
 - Examine **header fields** in all IP datagrams passing through it.
 - Move **datagrams from input ports to output ports** to transfer datagrams along **end-end path**.

Two Key Functions: The network layer performs two key functions:

1. **Forwarding** (Data Plane Function)
 - **Definition:** move packets from a router's input link to appropriate router output link.
 - **Analogy:** process of getting through a **single interchange**.
2. **Routing** (Control Plane Function)
 - **Definition:** determine route taken by packets from source to destination.
 - **Analogy:** process of **planning trip from source to destination**.
 - Uses **routing algorithms**.

The functionality of the network layer is split into two planes:

Plane	Function	Description
-------	----------	-------------

Data Plane	local, per-router function	Determines how datagram arriving on router input port is forwarded to router output port.
Control Plane	network-wide logic	Determines how datagram is routed among routers along end-end path from source host to destination host. There are two main approaches to the control plane: 1. Traditional Routing Algorithms: Implemented in routers. Individual routing algorithm components in each router interact. 2. Software-Defined Networking (SDN): Implemented in (remote) servers. A Remote Controller computes, installs forwarding tables in routers.

A service model defines what kind of delivery guarantees the network layer provides to the transport layer.

Service Model Examples

- **Individual datagrams:** guaranteed delivery, guaranteed delay with less than 40 msec delay.
- **Flow of datagrams:** in-order datagram delivery, guaranteed minimum bandwidth to flow.

Internet "Best Effort" Service Model

The **Internet** uses a **"best effort" service model**. It provides **no guarantees** on:

- Successful datagram delivery to destination.
- Timing or order of delivery.
- Bandwidth available to end-end flow.

Reflections on Best-Effort Service: The **simplicity of mechanism** has allowed the Internet to be **widely deployed**. Other factors that help its performance include **sufficient provisioning of bandwidth, replicated, application-layer distributed services** (datacenters, CDNs) connecting close to clients' networks, allow services to be provided from multiple locations, and **congestion control**.

Network-layer service model

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

5. Forwarding and Addressing

Destination-Based Forwarding and Longest Prefix Matching

- **Forwarding** is typically performed based on the **destination IP address**.

Destination Address Range (binary)	Link Interface
11001000 00010111 00010000 00000000 → 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 → 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 → 11001000 00010111 00011111 11111111	2
11001000 00010111 00010000 00000100 → 11001000 00010111 00010000 00000111	3

- **Longest prefix matching** is the rule used: when looking for a forwarding table entry, use the **longest address prefix that matches destination address**. If multiple entries match, it chooses the one with the longest prefix — i.e., the most specific match (the most bits in common starting from the left).
- **Implementation:** Often performed using **Ternary Content Addressable Memories (TCAMs)**, Hardware used in routers for very fast lookups. A **content addressable** approach means the address is retrieved in **one clock cycle**, regardless of table size.

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

11001000 00010111 00010110	10100001	which interface?
11001000 00010111 00011000	10101010	which interface?

Sidebar: Network Neutrality

Network Neutrality is a set of principles enforced by legal rules and policies, concerning:

- **Technical:** How an ISP should share/allocate its resources (e.g., packet scheduling).
- **Social/Economic:** Protecting **free speech** and encouraging **innovation, competition**.

The 2015 US FCC Order on Protecting and Promoting an Open Internet included three “**clear, bright line**” rules:

1. **No blocking** → cannot block lawful content.
2. **No throttling** → cannot slow specific content.
3. **No paid prioritization** → cannot make fast lanes for money.

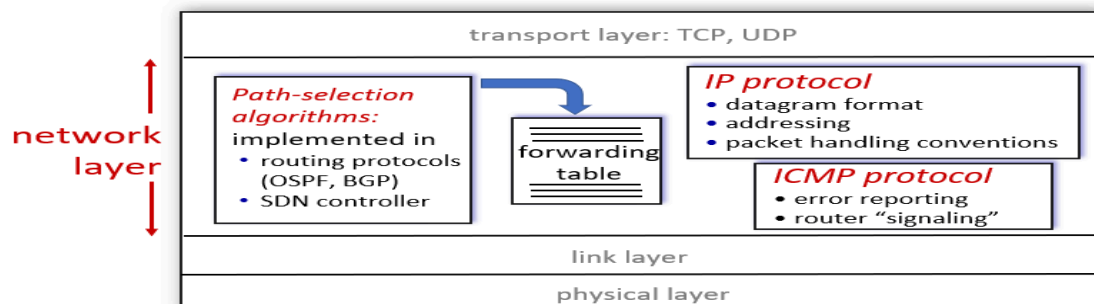
The core regulatory debate hinges on whether an **ISP** is a “**telecommunications service**” (**Title II**, regulated as a “**common carrier**”) or an “**information service**” (**Title I**, not regulated) under the US Telecommunication Act.

IP: The Internet Protocol

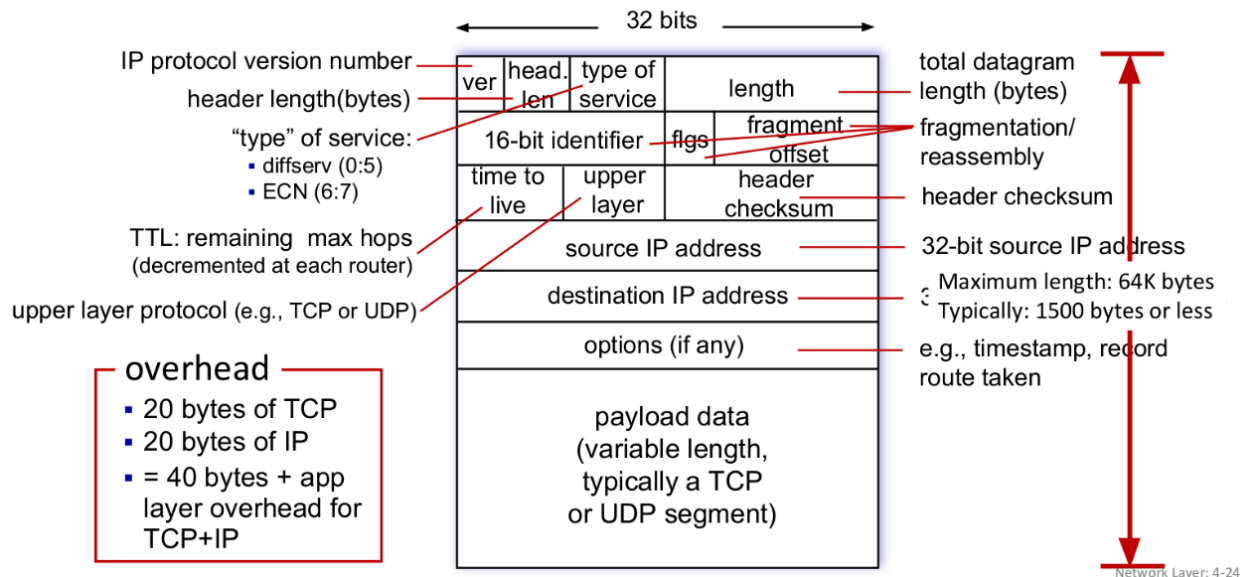
Network Layer (Internet): provides IP-based datagram forwarding, addressing, and error reporting (via ICMP), supporting TCP/UDP transport, with path selection handled by routing protocols (OSPF, BGP) or SDN controllers using forwarding tables between the link and physical layers.

Network Layer: Internet

host, router network layer functions:



IP Datagram format



The combined TCP and IP header overhead is typically **40 bytes**.

IP Fragmentation/Reassembly

- **Maximum Transmission Unit (MTU) (max. transfer size):** The largest possible link-level frame size.
- **Fragmentation:** Large IP datagrams are **divided ("fragmented") within the network** when they encounter a link with a smaller MTU. One datagram becomes several datagrams.
- **Reassembly:** Fragments are **reassembled only at the destination**.
- **Image:** Shows a 4000-byte datagram (with a 20-byte header, 3980 data bytes) fragmented due to a 1500-byte MTU (which allows 1480 data bytes). The **fragment offset** field is measured in units of 8 bytes:
 - Fragment 1: **offset = 0, length = 1500, fragflag = 1.**
 - Fragment 2: **offset = 185 (1480/8), length = 1500, fragflag = 1.**
 - Fragment 3: **offset = 370 (185 + 185), length = 1040, fragflag = 0** (last fragment).

IP Address and Interface

- **IP address:** A **32-bit identifier** associated with each **host or router interface**.
- **Interface:** A **connection between host/router and physical link**. Routers typically have multiple interfaces; hosts typically have one or two.
- **Notation:** Addresses are typically written in **dotted-decimal IP address notation** (e.g., 223.1.1.1).
- **Host IP assignment:** either manually configured by the system admin or automatically obtained from a DHCP server using plug-and-play dynamic addressing.

Subnets and CIDR

- **Subnet:** A subnet is a group of devices that can reach each other directly without going through a router.
- An IP address is structured into a **subnet part** (common high order bits for devices in the same subnet) and a **host part** (remaining low order bits).
- **CIDR (Classless InterDomain Routing):** The address format is **a.b.c.d/x**, where **x** is the # of bits in the subnet portion of the address. This allows the subnet portion to be of arbitrary length.

Dynamic Host Configuration Protocol (DHCP): DHCP allows a host to **dynamically obtain an IP address from a network server** when it joins a network (“**plug-and-play**”). Host can:

- renew its lease on address in use
- allows reuse of addresses (only hold address while connected/on)
- support for mobile users who join/leave network

Step	Message	Source/Destination	Description
1	DHCP discover	Host broadcasts (src: 0.0.0.0, dest: 255.255.255.255)	"is there a DHCP server out there?"
2	DHCP offer	Server broadcasts (src: Server IP, dest: 255.255.255.255)	Server offers an IP address (yiaddr).
3	DHCP request	Host broadcasts (src: 0.0.0.0, dest: 255.255.255.255)	Host accepts and requests the IP address.
4	DHCP ACK	Server broadcasts (src: Server IP, dest: 255.255.255.255)	Server confirms the IP address allocation.

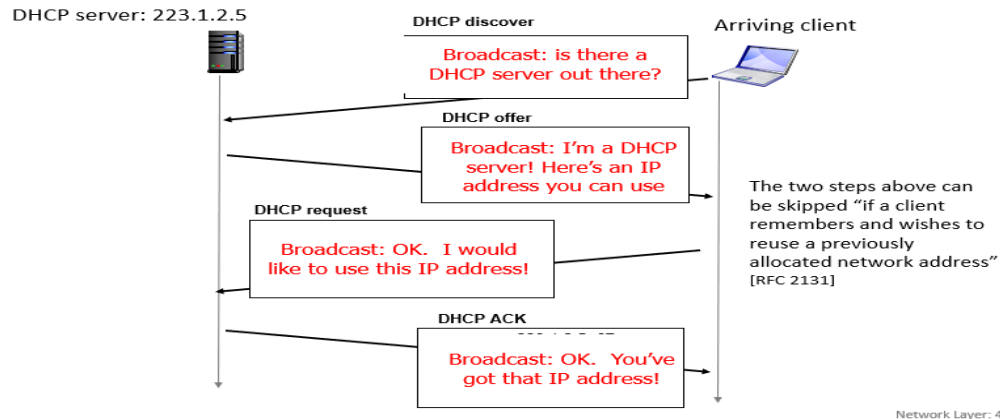
Configuration Details and Data Provided

In addition to dynamically obtaining an **IP address**, the DHCP process allows a client to receive critical network configuration parameters:

- The address of the **first-hop router** (the client's **default gateway**).
- The **name and IP address of a DNS server**.
- The **network mask** (or **subnet mask**).

This configuration is typically managed by a router that functions as the **DHCP server**.

DHCP client-server scenario



Message Encapsulation and Communication Flow

The DHCP process relies on network-layer broadcasting to locate the server:

- **Layering**: DHCP application-layer messages are encapsulated using the **UDP** transport layer and the **IP** network layer before being passed to the **Ethernet** link layer. The full stack is: **DHCP** → **UDP** → **IP** → **Ethernet**.
- **Broadcasting**: Each DHCP message (**Discover**, **Offer**, **Request**, **ACK**) is initially **broadcast** across the Local Area Network (LAN).
 - At the link layer, broadcasting uses the Ethernet destination address **FF:FF:FF:FF:FF:FF**.
 - At the network layer, broadcasting uses the IP destination address **255.255.255.255**.
- **Demultiplexing**: The messages are received by the router and demultiplexed layer by layer (**Ethernet** → **IP** → **UDP** → **DHCP**) to reach the DHCP server application.

Completion: Upon the client receiving the final **DHCP ACK** message, the client is **fully configured** with a valid IP address and essential network parameters, making it ready for full communication on the network.

Hierarchical Addressing and ICANN

IP addresses are assigned in blocks, not individually.

ICANN → Regional Internet Registries (RIRs)

RIRs → ISPs

ISPs → organizations or customers

- **Network Address Allocation**: A network gets its address space allocated as a portion of its **provider ISP's address space**.
- **Route Aggregation**: Hierarchical addressing allows for **efficient advertisement of routing information**. A major ISP can advertise one large address block (e.g., 200.23.16.0/20) that encompasses all the smaller blocks allocated to its organizations, which is called **route aggregation**.

- **More Specific Routes:** If an organization moves to a different ISP, the new ISP can advertise a **more specific route** (e.g., 200.23.18.0/23) to override the aggregate route.
- **ICANN (Internet Corporation for Assigned Names and Numbers):** The organization that **allocates IP addresses** (through five regional registries, (who may then allocate to local registries)) and manages the DNS root zone, including delegation of individual TLD (.com,.edu , ...) management
- IPv4's 32-bit address space was exhausted in 2011, prompting reliance on NAT for conservation and the introduction of IPv6 with a vastly larger 128-bit address space.

Network Address Translation (NAT): NAT allows all devices in a local network to **share just one IPv4 address** as far as the outside world is concerned. Devices inside the local network use “private” IP address space (e.g., 10/8).

Advantages

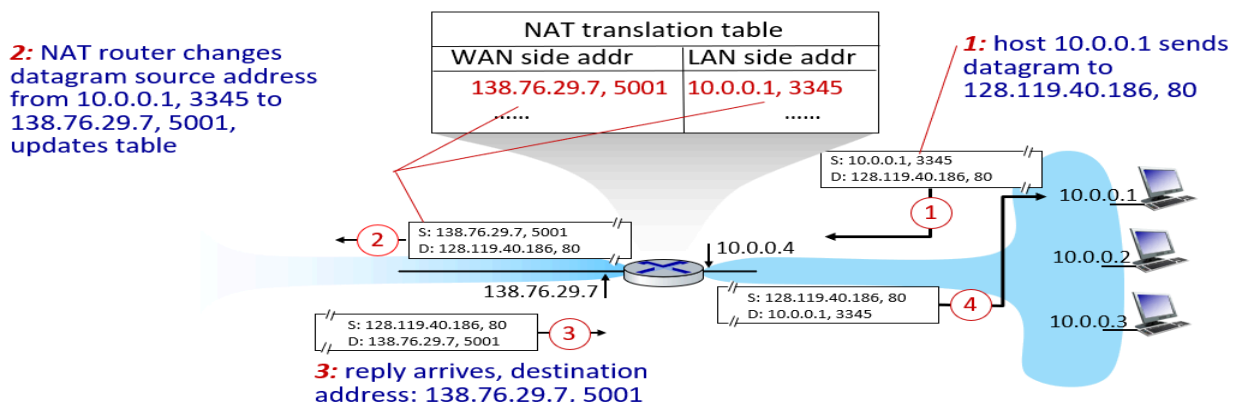
1. Only **one IP address needed from provider ISP** for all devices.
2. Can change addresses of hosts in the local network without notifying the outside world.
3. Can change ISP without changing addresses of devices in localnetwork
4. **Security:** Devices inside the local network are **not directly addressable, visible by the outside world**.

Implementation

The **NAT router** maintains a **NAT translation table** and performs address/port rewriting:

- **Outgoing datagrams:** Replace the original (**source IP address, port #**) with the (**NAT IP address, new port #**). Remote clients/servers will respond using (NAT IP address, new port #) as destination address. The translation is logged in the table.
- **Incoming datagrams:** Replace the (**NAT IP address, new port #**) in the destination field with the corresponding (**source IP address, port #**) stored in the table.

NAT: network address translation



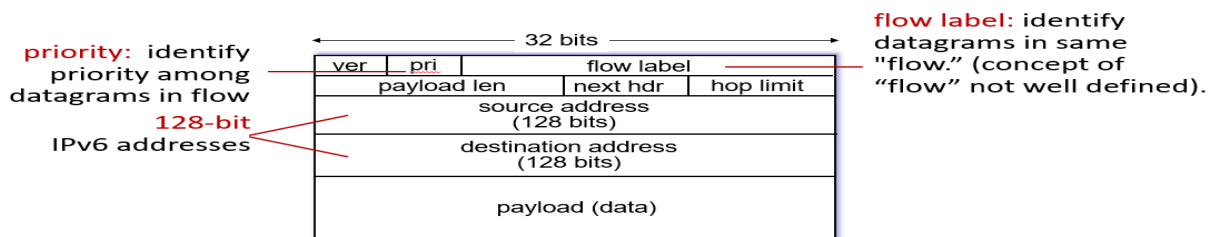
Controversy

NAT has been controversial as routers “should” only process up to layer 3, address “shortage” should be solved by IPv6, it violates the end-to-end argument, complicates NAT traversal, but is here to stay—extensively used in home, institutional, and 4G/5G cellular nets.

IPv6 Motivation

- **Address Space Exhaustion:** The initial motivation was that the 32-bit IPv4 address space would be completely allocated. **IPv6 has 128-bit address space.**
- **Processing Speed:** Additional motivation was: **speed processing/forwarding** via a **40-byte fixed length header** and flow-based network-layer treatment.

IPv6 datagram format



What's missing (compared with IPv4):

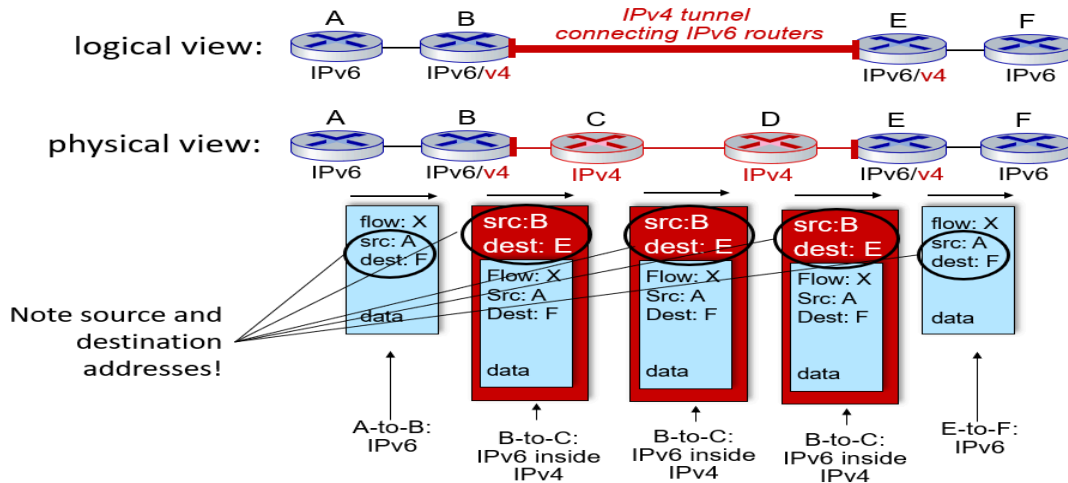
- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

Transition from IPv4 to IPv6: Tunneling

Since not all routers can be upgraded simultaneously (no “**flag days**”), a transition mechanism is needed.

- **Tunneling:** An **IPv6 datagram is carried as payload in an IPv4 datagram** among IPv4 routers, creating a “**packet within a packet**”.
- **Image Analysis (Slides 4-54, 4-55, 4-56):** The diagrams illustrate this. An IPv6 datagram travels normally between IPv6-capable routers (A to B) and (E to F). To cross an IPv4-only network segment (B to E), router B encapsulates the entire IPv6 datagram as the payload of a new IPv4 datagram. Router E removes the IPv4 header before forwarding the original IPv6 datagram to F.

Tunneling



IPv6 adoption remains slow (~40% global, 1/3 US gov domains) despite 25 years, due to complex deployment, legacy IPv4 compatibility, and rapid app-level evolution outpacing network upgrades.

Generalized Forwarding

Generalized forwarding follows a “match + action” model: the switch matches specific header fields in an incoming packet and performs an action (forward, drop, modify, etc.).

These match–action rules are stored in flow tables, which give modern networks flexibility.

In SDN, the OpenFlow protocol lets the controller dynamically install or update these flow-table entries.

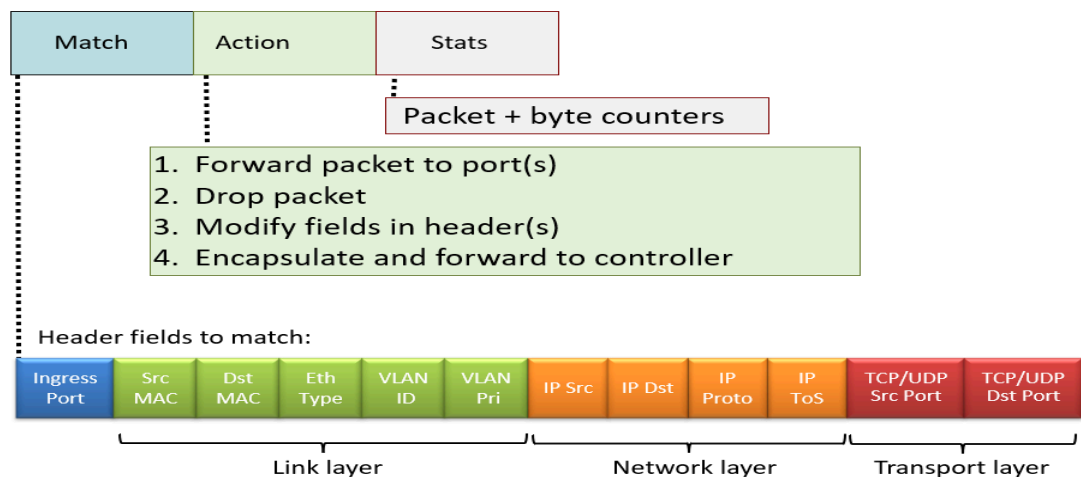
- **Match:** Can be over many header fields (link-, network-, and transport-layer fields).
- **Action:** Possible actions include drop, forward, modify, copy, log, or send the matched packet to the controller.
- **Priority:** Resolves overlapping match patterns. Selects the more specific rule
- **Counters:** Record the number of matched bytes and packets.
- **Forwarding Table (Flow Table):** Defines the router’s *match + action* rules; a generalized form of the traditional routing table.
- In **destination-based forwarding**, routers match only the destination IP address to choose an output port.
- In **generalized forwarding**, the flow table can match on multiple fields (e.g., source/destination IPs, port numbers, protocols) and take many possible actions.
- **Flow:** Defined by specific header field values from link, network, and transport layers. A set of packets that share header values

Traditional Router Forwarding Table	SDN Flow Table
Matches only destination IP	Matches many fields (IP, MAC, ports, protocol, etc.)
Action is only forward to next hop/port	Actions include forward, drop, modify header, send to controller
Used only for IP routing	Used for routing, switching, firewall, NAT, QoS
Fixed-function hardware	Programmable and flexible
Control logic is distributed (each router decides)	Control logic is centralized in SDN controller
Slow to change (manual configuration)	Fast, dynamic updates via OpenFlow

OpenFlow is a concrete protocol that implements the **match+action** flow table abstraction. It allows matching on a wide range of header fields:

- **Link Layer:** Source/Dst MAC, Eth Type, VLAN ID/Pri.
- **Network Layer:** IP ToS, IP Proto, IP Src/Dst.
- **Transport Layer:** TCP/UDP Src/Dst Port.

OpenFlow: flow table entries



Unifying Device Types

The **match+action** abstraction can describe the functions of many different network devices:

- **Router:** match: **longest destination IP prefix**; action: forward out a link.
- **Switch:** match: **destination MAC address**; action: forward or flood.

- **Firewall:** match: **IP addresses and TCP/UDP port numbers**; action: permit or deny.
- **NAT:** match: **IP address and port**; action: rewrite address and port.

Network-Wide Behavior: Orchestrated tables (set by a controller) can create complex, network-wide behaviors.

“Match plus action” abstraction matches bits across any header fields to take actions (drop, forward, modify, send to controller), enabling programmable, per-packet network behavior—an early form of network programmability now extended by P4.

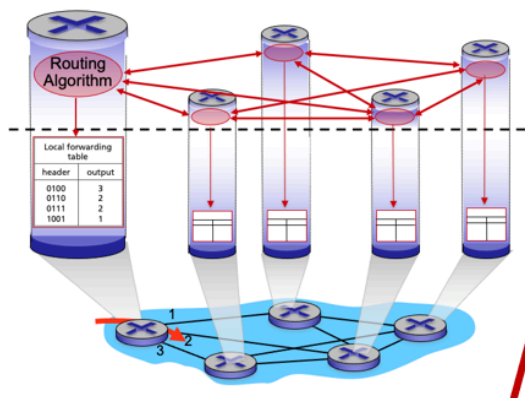
Chapter 5: Network Layer - Control Plane

- Two approaches to structuring network control plane:
 - per-router control (traditional)
 - logically centralized control (software defined networking)
- Network-layer functions:
 - forwarding: move packets from router's input to appropriate router output (Data Plane)
 - routing: determine route taken by packets from source to destination (Control Plane)

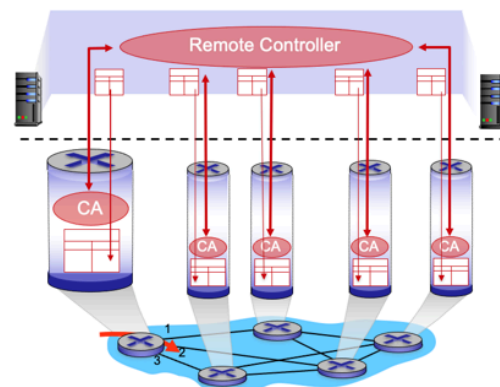
Per Router Control Plane: Individual routing algorithm components in each and every router interact in the control plane.

Software-Defined Networking (SDN) control plane: Remote controller computes, installs forwarding tables in routers.

Per-router control plane



SDN control plane

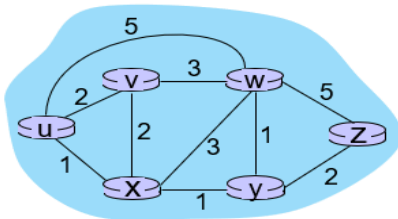


Routing Protocols:

- **Routing protocol goal:** determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers.

- **path**: sequence of routers packets traverse from given initial source host to final destination host.
- **“good”**: least **“cost”**, **“fastest”**, **“least congested”**.
- routing: a “top-10” networking challenge!

Graph abstraction: link costs



$c_{a,b}$: cost of **direct** link connecting a and b
e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator:
 could always be 1, or inversely related to bandwidth, or inversely related to congestion

graph: $G = (N, E)$

N : set of routers = $\{ u, v, w, x, y, z \}$

E : set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Directly proportional to congestion*

Routing algorithm classification

- **Classification 1: Information distribution (Source of data):**
 - **Global**: All routers have **complete topology** and **link cost info**. Used by “**link state**” algorithms.
 - **Decentralized**: **Iterative process** exchanging info with **neighbors**. Routers initially only know link costs to **attached neighbors**. Used by “**distance vector**” algorithms.
- **Classification 2: Route change frequency (Dynamics):**
 - **Dynamic**: Routes change more quickly, based on **periodic updates** or response to **link cost changes**.
 - **Static**: Routes change slowly.

Dijkstra’s Link-State Routing Algorithm

- **Type**: Global / centralized
- **Information**: All nodes know network topology and link costs (via **link state broadcast**)
- **Goal**: Compute **least-cost paths** from a source node to all other nodes
- **Output**: Forwarding table for the source node
- **Process**: Iterative
 - After k iterations $\rightarrow$ know least-cost path to k destinations

Notation:

Symbol	Meaning
C_{xy}	Direct link cost from node x to y; = ∞ if not neighbors
$D(v)$	Current estimate of least-cost path from source to destination v
$p(v)$	Predecessor node along path from source to v
N'	Set of nodes whose least-cost paths are definitively known

Dijkstra's link-state routing algorithm

```

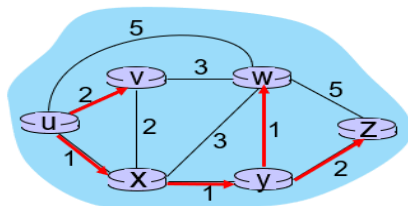
1 Initialization:
2  $N' = \{u\}$  /* compute least cost path from u to all other nodes */
3 for all nodes v
4   if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */
5     then  $D(v) = c_{u,v}$  /* but may not be minimum cost! */
6   else  $D(v) = \infty$ 
7
8 Loop
9   find w not in  $N'$  such that  $D(w)$  is a minimum
10  add w to  $N'$ 
11  update  $D(v)$  for all v adjacent to w and not in  $N'$ :
12     $D(v) = \min ( D(v), D(w) + c_{w,v} )$ 
13  /* new least-path-cost to v is either old least-cost-path to v or known
14     least-cost-path to w plus direct-cost from w to v */
15 until all nodes in  $N'$ 

```

Network Layer: 5-14

Dijkstra's algorithm: an example

Step	N'	$D(v)$ $p(v)$	$D(w)$ $p(w)$	$D(x)$ $p(x)$	$D(y)$ $p(y)$	$D(z)$ $p(z)$
0	u	2 u	5 u	1 u	∞	∞
1	u, x	2 u	4 x		2 x	∞
2	u, x, y	2 u	3 y			4 y
3	u, x, y, v		3 y			4 y
4	u, x, y, v, w					4 y
5	u, x, y, v, w, z					



Initialization (step 0): For all a: if a adjacent to then $D(a) = c_{u,a}$

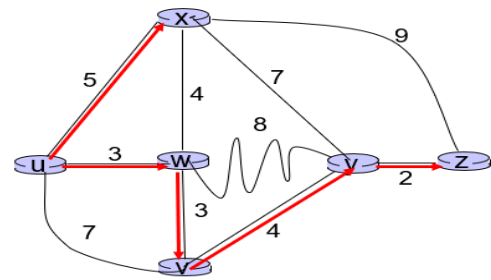
```

find a not in  $N'$  such that  $D(a)$  is a minimum
add a to  $N'$ 
update  $D(b)$  for all b adjacent to a and not in  $N'$ :
 $D(b) = \min ( D(b), D(a) + c_{a,b} )$ 

```

Dijkstra's algorithm: another example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7, u	3, u	5, u	∞	∞
1	uw	6, w		5, u	11, w	∞
2	uwX	6, w			11, w	14, x
3	uwXv				10, v	14, x
4	uwXvy					12, y
5	uwXvyZ					



notes:

- construct least-cost-path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

Dijkstra's Algorithm: Discussion

1. Algorithm Complexity

- Nodes:** n
- Iterations:** For each of n iterations, need to check all nodes w not in N'
- Comparisons:** $n(n+1)/2 \rightarrow O(n^2)$ complexity
- Optimized Implementations:** Using priority queues / heaps $\rightarrow O(n \log n)$

2. Message Complexity

- Each router broadcasts its **link state information** to all other n routers
- Efficient broadcast algorithms:**
 - Send out a message from one source using $O(n)$ link crossings
- Overall message complexity:** Each router's message crosses $O(n)$ links $\rightarrow$ total $O(n^2)$

Oscillations in Dijkstra's Algorithm: Routing changes back and forth, never stable

- Cause:** Link costs depend on **traffic volume** $\rightarrow$ possible **route oscillations**
- Scenario Example:**
 - Destination: a
 - Traffic entering network at d, c, e with rates: 1, e (<1), 1
 - Link costs are directional and volume-dependent**
- Effect:**
 - Initial link costs $\rightarrow$ compute new routing $\rightarrow$ update link costs $\rightarrow$ recompute routing
 - Can result in **cyclical updates**, causing oscillations in path selection
- Observation:** Repeated updates of routing based on dynamic, traffic-dependent costs may lead to **unstable routes** if costs constantly change.

Distance Vector DV Algorithm

1. Bellman-Ford (BF) Equation (Dynamic Programming)

- **Basis:** DV algorithm is based on the Bellman-Ford equation.
- **Notation:** $D_x(y)$ = cost of least-cost path from node x to destination y .
- **Equation:**

$$D_x(y) = \min_v \{c_{x,v} + D_v(y)\}$$

- $\min_v$: minimum over all neighbors v of x
- $c_{x,v}$: direct cost of link from x to v
- $D_v(y)$: neighbor v 's estimated least-cost-path cost to y

Bellman-Ford Example

- **Scenario:** Suppose node u wants to find the least-cost path to destination z . u 's neighbors (x, v, w) have the following estimated least-cost path costs to z : $D_v(z) = 5$, $D_w(z) = 3$, and $D_x(z) = 3$.

- *The image context (links and costs):*

- Link (u, v) cost: 2.
- Link (u, x) cost: 1.
- Link (u, w) cost: 5.

- **Calculation:** The BF equation for $D_u(z)$ is:

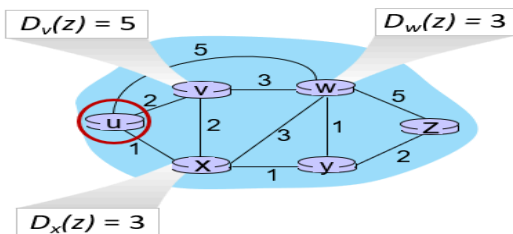
$$D_u(z) = \min\{c_{u,v} + D_v(z), \quad c_{u,x} + D_x(z), \quad c_{u,w} + D_w(z)\}$$

$$D_u(z) = \min\{2 + 5, \quad 1 + 3, \quad 5 + 3\} = 4$$

- **Result:** The node achieving minimum (x with cost 4) is the next hop on estimated least-cost path to destination (z).

Bellman-Ford Example

Suppose that u 's neighboring nodes, x, v, w , know that for destination z :



Bellman-Ford equation says:

$$\begin{aligned} D_u(z) &= \min \{ c_{u,v} + D_v(z), \\ &\quad c_{u,x} + D_x(z), \\ &\quad c_{u,w} + D_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

node achieving minimum (x) is next hop on estimated least-cost path to destination (z)

Distance Vector Algorithm Mechanism

Key Idea

- Nodes periodically **send their DV estimates** to neighbors.
- Upon receiving a DV update from neighbor v , node x updates its own DV using BF equation:

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\}, \quad \forall y \in N$$

- **Convergence:** Under typical conditions, $D_x(y)$ converges to actual least-cost $d_x(y)$.

Operational Flow (Per Node)

- **Iterative & Asynchronous:** Each local iteration triggered by:
 - **Change in local link cost**
 - **DV update from a neighbor**
- **Steps:**
 - Wait for change in local link cost or message from neighbor
 - Recompute DV estimates using received neighbor DVs
 - If DV to any destination changes → notify neighbors
- **Distributed & Self-Stopping:**
 - Notify neighbors **only when DV changes**
 - Neighbors propagate updates **only if necessary**
 - **No update received** → no action taken

SEE SLIDES FOR COMPUTATION EXAMPLE

Aspect	Link-State (LS)	Distance Vector (DV)
Message Complexity	n routers, $O(n^2)$ messages sent	Exchange between neighbors ; convergence time varies
Speed of Convergence	$O(n^2)$ algorithm, $O(n^2)$ messages; may have oscillations	Convergence time varies ; may have routing loops and count-to-infinity problem
Robustness / Fault Tolerance	If a router malfunctions or is compromised: can advertise incorrect link cost; each router computes only its own table , limiting error propagation	If a DV router advertises incorrect path cost (e.g., "low-cost path to everywhere"), it can black-hole traffic ; errors propagate through the network since each router's DV is used by neighbors

Making Routing Scalable

Idealized Routing vs. Reality

An **Idealized Study** might assume all routers are identical and the network is "flat."

In **Reality**, this is impractical due to two main issues:

1. **Scale:** With billions of destinations, routers **cannot store all destinations in routing tables!** The exchange of all routing tables would **swamp links!**
2. **Administrative Autonomy:** The Internet is a "network of networks." Each network administrator may want to **control routing within their own network.**

Internet Approach to Scalable Routing

The Internet achieves scalability through **Aggregation**: Routers are aggregated into regions known as **autonomous systems (AS)** (also called "domains").

This leads to two distinct routing approaches:

- **Intra-AS (aka "intra-domain"):**
 - Routing occurs **among routers within the same AS** ("network").
 - **All routers in an AS must run the same intra-domain protocol**, but routers in different ASes can run different ones.
 - A **Gateway router** sits at the "edge" of its AS, having link(s) to router(s) in other ASes.
- **Inter-AS (aka "inter-domain"):**
 - Routing occurs **among ASes**.
 - **Gateways perform inter-domain routing** (as well as intra-domain routing).

Interconnected ASes (Forwarding Table Configuration)

The **Forwarding Table** is configured by both the intra- and inter-AS routing algorithms:

- **Intra-AS routing** determines entries for destinations **within the AS**.
- **Inter-AS & Intra-AS** together determine entries for **external destinations**.

Inter-AS Routing: Role in Intra-domain Forwarding

For an AS, such as AS1:

1. **AS1 Inter-domain routing must:**
 - Learn which destinations are reachable through AS2, and which through AS3.
 - Propagate this reachability information to all routers in AS1.
2. **Packet Handling:** If a router in AS1 receives a datagram destined **outside of AS1**, it must forward the packet to the correct gateway router in AS1 (which one?).

Intra-AS Routing Protocols

Most Common Protocols

Protocol	Type	Description
RIP (Routing Information Protocol) [RFC 1723]	Classic DV (Distance Vector)	DVs exchanged every 30 seconds. No longer widely used.
EIGRP (Enhanced Interior Gateway Routing Protocol)	DV based	Formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868]).
OSPF (Open Shortest Path First) [RFC 2328]	Link-state routing	Widely Used (see details below).
IS-IS protocol (ISO standard)	Link-state routing	Essentially the same as OSPF.

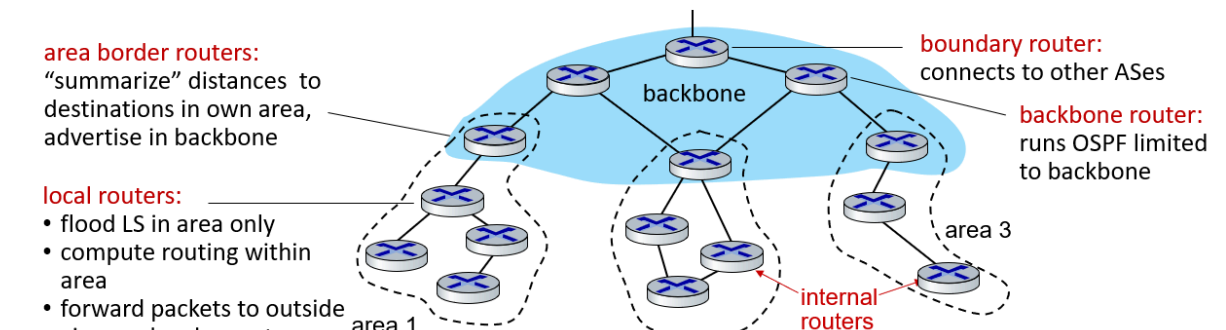
OSPF (Open Shortest Path First) Routing

- **"Open"**: Publicly available.
- **Protocol Type**: Classic link-state.
- **LSA Flooding**: Each router floods OSPF **link-state advertisements** (LSA) to all other routers in the entire AS. (This is done directly over IP, not using TCP/UDP).
- **Computation**: Each router has the **full topology** and uses **Dijkstra's algorithm** to compute its forwarding table.
- **Metrics**: Multiple link cost metrics are possible (e.g., bandwidth, delay).
- **Security**: All OSPF messages are **authenticated** to prevent malicious intrusion.

Hierarchical OSPF

OSPF can be structured into a **two-level hierarchy**: local area and backbone (area 0 only).

- **LSA Scope**: Link-state advertisements are flooded **only in an area, or the backbone**.
- **Topology Knowledge**: Each node has detailed area (multiple areas inside AS) topology, but only knows the **direction** to reach destinations outside its area. All areas must connect to area 0.
- **Router Roles**:
 - **Local routers**: Flood LS in area only, compute routing within the area, and forward packets to the outside backbone.
 - **Area border routers**: **"Summarize"** distances to destinations in their own area and advertise this summary in the backbone.
 - **Backbone router**: Runs OSPF limited to the backbone.
 - **Boundary router**: Connects to other ASes.



BGP (Border Gateway Protocol)

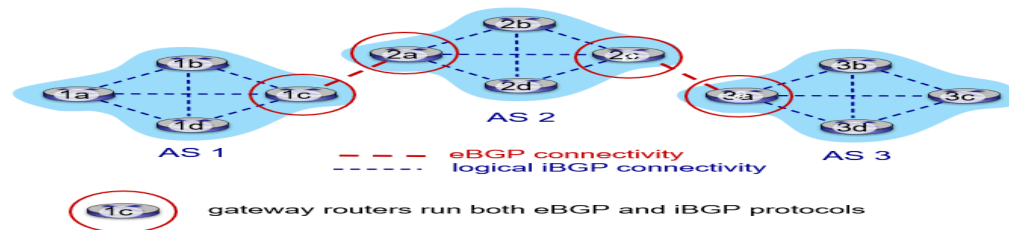
Role: BGP is the **in practise inter-domain routing protocol**—the **"glue that holds the Internet together."**

Function: It allows a subnet to advertise its existence and the destinations it can reach: "I am here, here is who I can reach, and how."

BGP Provides Each AS a Means to:

1. Obtain destination network **reachability info from neighboring ASes** (eBGP).
2. Determine routes to other networks based on the reachability info and **policy**.
3. Propagate reachability information to all AS-internal routers (iBGP).
4. Advertise destination reachability info to neighboring networks.

eBGP, iBGP connections



BGP Basics

- **BGP Session:** Two BGP routers ("**peers**") exchange BGP messages over a **semi-permanent TCP connection**.
- **Protocol Type:** BGP is a "**path vector**" **protocol**, advertising **paths to different destination network prefixes**. Path Vector means here is the path of ASes to reach destination.
- **Forwarding Promise:** When AS3 gateway 3a advertises path AS3, X to AS2 gateway 2c, AS3 **promises to AS2 it will forward datagrams towards X**. In short, when an AS advertises a route, it promises it will forward packets along that route.

BGP Protocol Messages

BGP messages are exchanged between peers over the TCP connection:

- **OPEN:** Opens the TCP connection and authenticates the sending BGP peer.
- **UPDATE:** Advertises a new path (or withdraws an old one).
- **KEEPALIVE:** Keeps the connection alive in the absence of UPDATES; also ACKs the OPEN request.
- **NOTIFICATION:** Reports errors or closes the connection.

Path Attributes and BGP Routes

A BGP **Advertised Route** consists of a **prefix** (the destination being advertised) and **attributes**. Prefix advertisement means: "I can deliver packets to this IP Block"

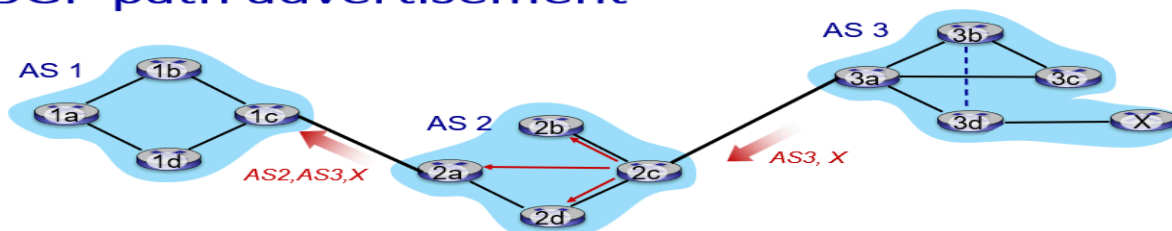
Two Important Attributes:

1. **AS-PATH:** A list of ASes through which the prefix advertisement has passed.
2. **NEXT-HOP:** Indicates the specific internal-AS router that is the next-hop to the next-hop AS. (immediate router to forward to)

Policy-Based Routing:

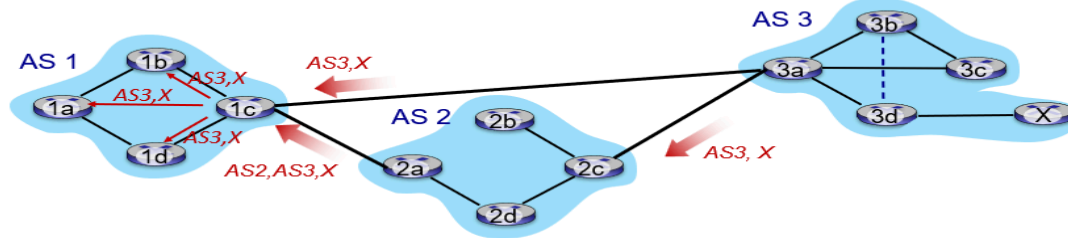
- A gateway uses an **import policy** to accept or decline a path (e.g., "never route through AS Y").
- AS policy also determines whether to advertise the path to other neighboring ASes.

BGP path advertisement



- AS2 router 2c receives path advertisement **AS3,X** (via eBGP) from AS3 router 3a
- based on AS3 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all

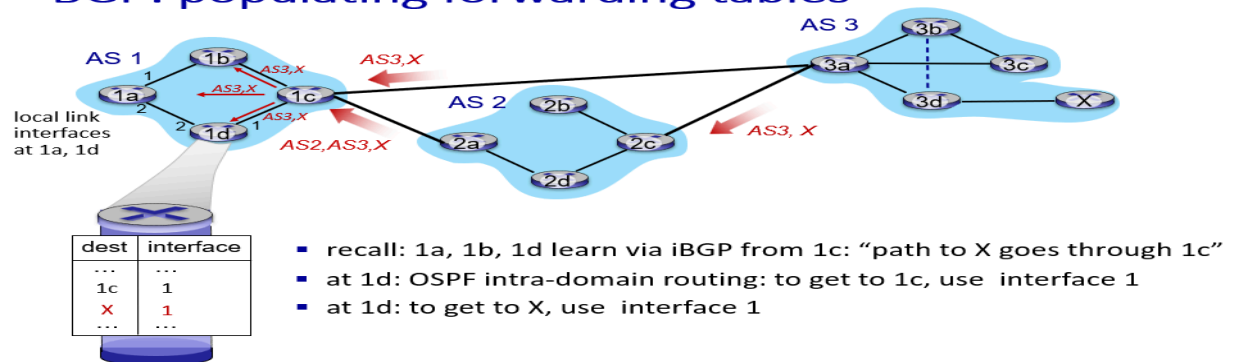
BGP path advertisement: multiple paths



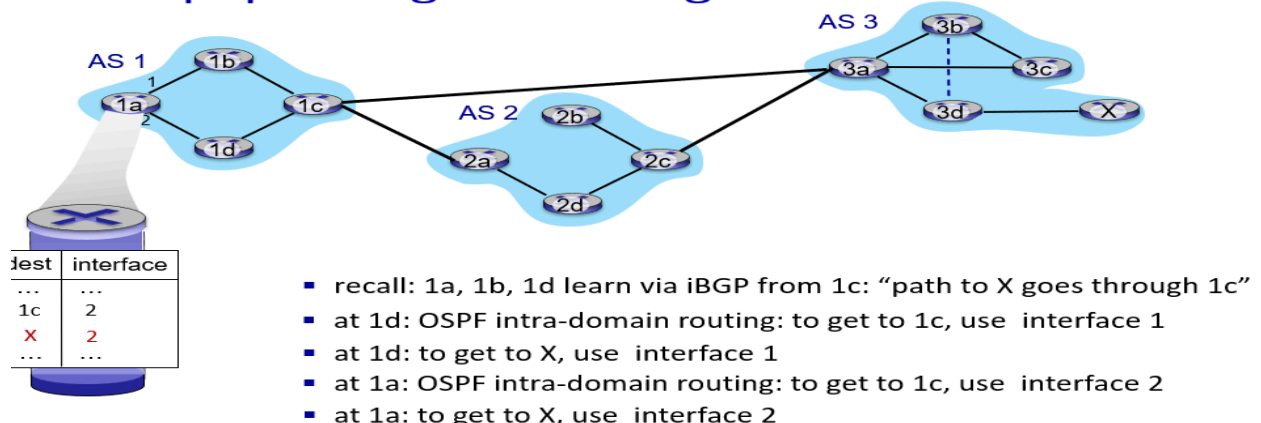
gateway router may learn about **multiple** paths to destination:

- AS1 gateway router 1c learns path **AS2,AS3,X** from 2a
- AS1 gateway router 1c learns path **AS3,X** from 3a
- based on **policy**, AS1 gateway router 1c chooses path **AS3,X** and advertises path within AS1 via iBGP

BGP: populating forwarding tables



BGP: populating forwarding tables



Hot Potato Routing

- **Decision:** When a router (e.g., 2d) learns it can route to X via multiple gateways (e.g., 2a or 2c), it uses **Hot Potato Routing**. If there are multiple exit gateways to the same destination, it chooses the one which is closest.
- **Mechanism:** Choose the **local gateway** that has the **least intra-domain cost** (measured by OSPF link weights).
- **Principle:** "**Don't worry about inter-domain cost!**" — Get the packet out of your AS as quickly as possible.

BGP: Achieving Policy via Advertisements

- **Typical ISP Policy:** An ISP only wants to route traffic **to/from its customer networks**; it **does not want to carry transit traffic** between other ISPs.
- **Enforcement Example:**
 - Provider A advertises path A, w to B and C.
 - Provider B **chooses not to advertise** path B, A, w to C because B gets **no "revenue"** for routing transit traffic C to B to A to w.
 - This forces C to route C to A to w (not using B).
- **Customer Example (Dual-homed x):** If customer x is attached to providers B and C, x typically has a policy not to route traffic from B to C via x. To enforce this, **x will not advertise to B a route to C**.

BGP Route Selection

- A router may learn multiple routes to a destination AS and selects the route based on the following ordered criteria:
 1. **Local preference value attribute** (the primary **policy decision**).
 2. **Shortest AS-PATH**.
 3. **Closest NEXT-HOP router** (determined by **hot potato routing**).
 4. **Additional criteria**.

Why Different Intra-, Inter-AS Routing?

Criterion	Inter-AS (BGP)	Intra-AS (OSPF, RIP, etc.)
Policy	Dominates. Admin wants control over how its traffic is routed and who routes through its network.	Less of an issue because there is a single admin.
Scale	Hierarchical routing (ASes) saves table size and results in reduced update traffic. Without AS hierarchy, every router would need routes to billions of prefixes	Less of a driving factor.

Performance	Policy dominates over performance.	Can focus on performance (e.g., finding the mathematically shortest path).
--------------------	------------------------------------	--

Software Defined Networking (SDN)

I. Traditional Internet Network Layer (Per-Router Control Plane)

The Internet network layer was **historically implemented** using a **distributed, per-router control approach**:

- **Monolithic Router:** Contains switching hardware and runs a proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in a proprietary router OS (e.g., Cisco IOS).
- **Middleboxes:** Different specialized boxes exist for different network layer functions (e.g., firewalls, load balancers, NAT boxes).
- **Control Plane:** Individual routing algorithm components in each and every router interact to compute local forwarding tables.

In short routers run routing algorithms (control plane) that compute forwarding tables and the switching hardware (data plane) uses those tables to forward packets, while middleboxes independently enforce extra functions like security and load balancing.

Control Plane (Per-router)	Data Plane
Routing Algorithm component in each router	Local forwarding table used to map arriving packet header values (e.g., 0111) to an output port.

This led to a renewed interest in rethinking the network control plane around 2005.

II. Software-Defined Networking (SDN) Control Plane

SDN separates the control logic from the physical forwarding devices.

- **Remote Controller:** A logically centralized entity that computes and installs forwarding tables in the routers (now called SDN-controlled switches).

Control Plane (SDN)	Data Plane
Remote Controller	CA (Control Agent) on each router receives forwarding table updates.

III. Why a Logically Centralized Control Plane?

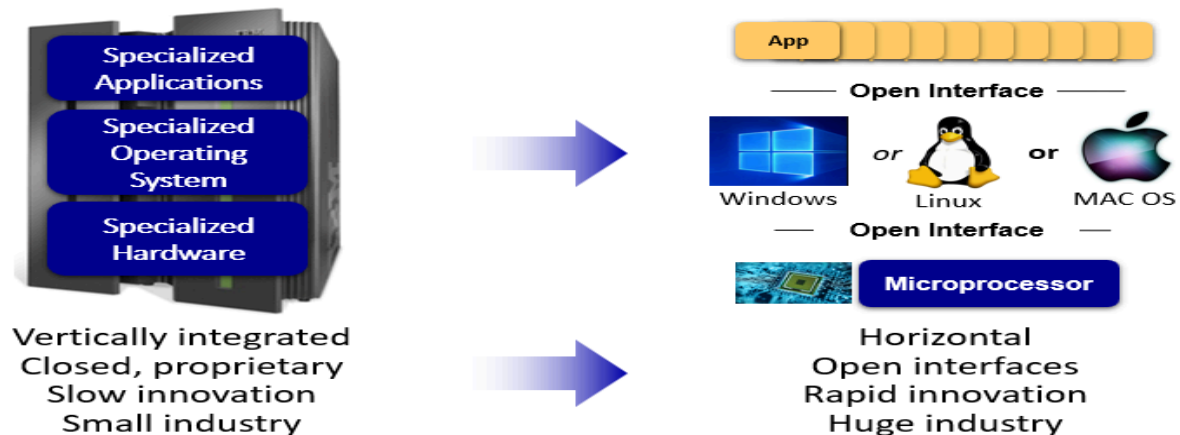
- **Easier Network Management:** Avoids router misconfigurations and offers greater flexibility of traffic flows.
- **"Programming" Routers:** Table-based forwarding (recall OpenFlow API) allows for "programming" routers.
 - **Centralized "Programming" is Easier:** Compute tables centrally and distribute them.
 - **Distributed "Programming" is More Difficult:** Compute tables as a result of a complex distributed algorithm/protocol implemented in each-and-every router.

- **Open (Non-Proprietary) Implementation of Control Plane:** Fosters innovation ("let 1000 flowers bloom").

SDN Analogy: Mainframe to PC Revolution

SDN parallels the shift from vertically integrated, closed systems to horizontal, open systems.

Vertically integrated (Traditional)	Horizontal (SDN)
Specialized Applications, OS, Hardware	App (Specialized) over Open Interface over Windows/Linux/MAC OS over Microprocessor
Closed, proprietary, slow innovation, small industry	Open interfaces, rapid innovation, huge industry



IV. Traffic Engineering: Difficulties with Traditional Routing

Traditional, destination-based routing (like LS, DV) is severely limited in its control capabilities:

Scenario	Traditional Routing Difficulty
Path Selection: Operator wants traffic u to z to flow along uvwz instead of uxyz.	Limited Control: Link weights are the only control "knobs," often insufficient. Redefining weights or creating a new algorithm is required.
Load Balancing: Operator wants to split u to z traffic along uvwz and uxyz.	Impossible: Cannot do this (or requires a new routing algorithm).
Traffic Differentiation: Router w wants to route blue and red traffic differently to z.	Impossible: Cannot do this with destination-based forwarding (and LS, DV routing).

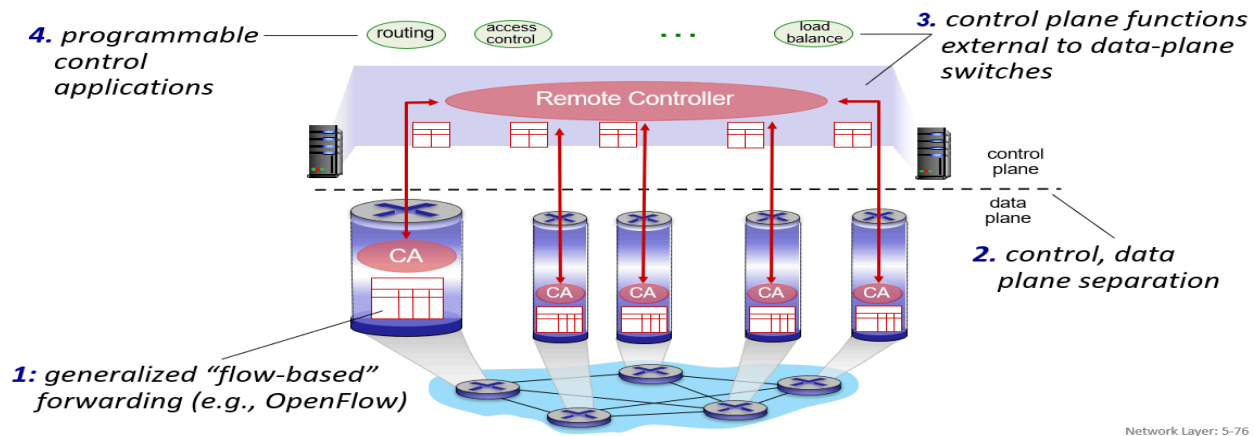
Generalized forwarding and SDN can be used to achieve any desired routing.

V. SDN Architecture and Components

SDN is defined by four key aspects: Programmable, centralised brain, simple switches, open interfaces

1. **Generalized "Flow-Based" Forwarding** (e.g., OpenFlow).
2. **Control, Data Plane Separation.**
3. **Control Plane Functions External to Data-Plane Switches.**
4. **Programmable Control Applications** (routing, access control, load balance, etc.).

Software defined networking (SDN)



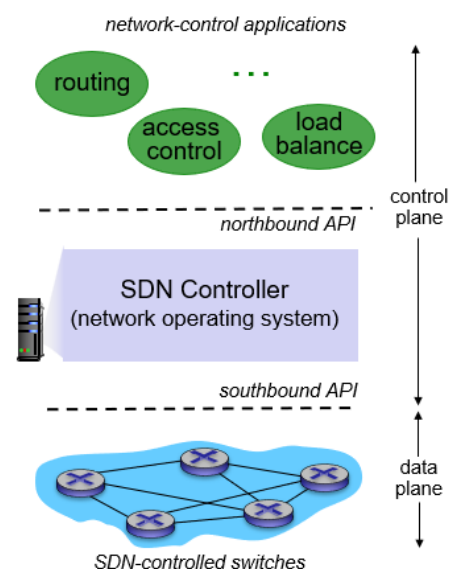
1. Data-Plane Switches

- **Function:** Fast, simple, commodity switches implementing generalized data-plane forwarding (Section 4.4) in hardware.
- **Flow Table:** Computed and installed under controller supervision.
- **API:** Provides an API for table-based switch control (e.g., OpenFlow), which defines what is and is not controllable.
- **Protocol:** Uses a protocol for communicating with the controller (e.g., OpenFlow).

2. SDN Controller (Network OS)

- **Function:** The "network operating system" that maintains network state information.
- **Interfaces:**
 - Interacts with network control applications **"above"** via the **Northbound API**.
 - Interacts with network switches **"below"** via the **Southbound API**.
- **Implementation:** Implemented as a distributed system for performance, scalability, fault-tolerance, and robustness.

3. Network-Control Applications

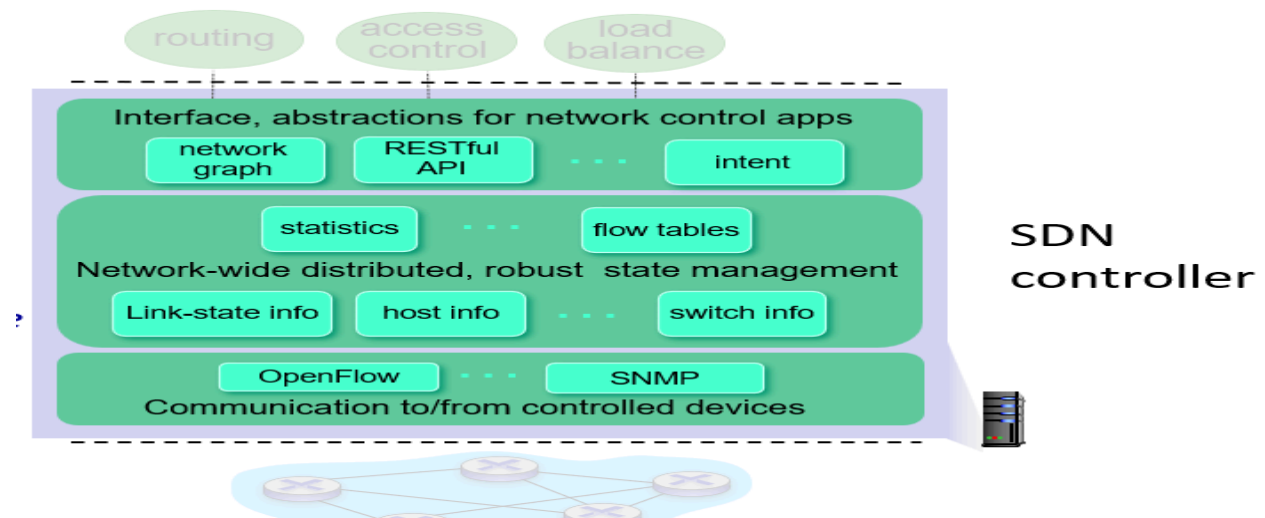


- **Function:** The "brains" of control; they implement control functions (e.g., routing, access control, load balance) using lower-level services and APIs provided by the SDN controller.
- **Nature:** They are **unbundled** and can be provided by a 3rd party, distinct from the routing vendor or the SDN controller.

4. Components of the SDN Controller (Internal Structure)

The controller has three main components:

- **Interface Layer to Network Control Apps:** Provides abstractions and an API (e.g., RESTful API, network graph, intent, flow tables) to the applications.
- **Network-Wide State Management:** A distributed database that maintains the state of network links, switches, services, host info, link-state info, and statistics.
- **Communication:** Handles communication to/from controlled devices using protocols like **OpenFlow** and **SNMP**.



VI. OpenFlow Protocol

OpenFlow is the core Southbound API protocol:

- **Operation:** Operates between the controller and the switch using **TCP** to exchange messages (with optional encryption).
- **Classes of Messages:** Three classes of messages are exchanged:
 1. **Controller-to-Switch**
 2. **Asynchronous** (switch to controller)
 3. **Symmetric** (miscellaneous)
- **Distinction:** It is distinct from the OpenFlow API, which is used to specify generalized forwarding actions/rules

Key OpenFlow Messages

Message Type	Direction	Key Messages and Function
--------------	-----------	---------------------------

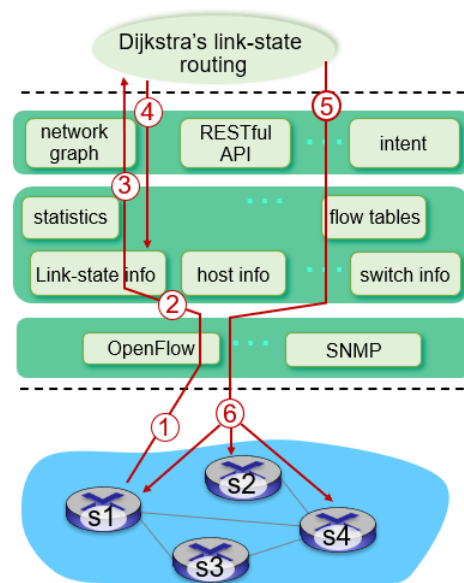
Controller-to-Switch	Controller-to-Switch	features : Controller queries switch features, switch replies.
		configure : Controller queries/sets switch configuration parameters.
		modify-state : Add, delete, or modify flow entries in the OpenFlow tables.
		packet-out : Controller sends a packet out of a specific switch port.
Asynchronous	Switch-to-Controller	packet-in : Transfers a packet (and its control) to the controller.
		flow-removed : Informs controller when a flow table entry is deleted at the switch.
		port status : Informs the controller of a change on a port (e.g., link failure).

Note: Network operators typically use a higher-level abstraction at the controller, rather than programming switches directly by sending OpenFlow messages.

VII. SDN Control/Data Plane Interaction Example (Dijkstra's Routing)

This example illustrates how the components interact in response to a link failure:

1. **Link Failure:** Switch s1, experiencing a link failure, uses an **OpenFlow port status** message to notify the controller.
2. **State Update:** The SDN controller receives the OpenFlow message and updates its **link status info** in the network-wide state management database.
3. **Application Call:** The **Dijkstra's routing algorithm application** (which previously registered to be called when link status changes) is invoked.
4. **Route Computation:** The routing application accesses the **network graph info** and **link state info** in the controller and computes new routes.
5. **Flow Table Computation:** The link state routing app interacts with a **flow-table-computation component** in the controller, which computes the new flow tables needed.
6. **Installation:** The controller uses **OpenFlow** (modify-state messages) to install the new tables in the switches that require updating.



VIII. Specific SDN Controllers/Architectures

ONOS SDN Controller

ONOS is an early, influential **open-source SDN controller**.

- **Architecture:** Control applications (e.g., Traffic Engineering, Firewalling, Load Balancing) are **separate from the controller core**.
- **Northbound API:** Includes an **Intent framework**, allowing for a high-level specification of service ("what" rather than "how").
- **Core:** A **distributed core** is emphasized for service reliability, replication performance, and scaling.
- **Southbound API:** Uses abstractions/protocols like OpenFlow, Netconf, and OVSDB to communicate with devices.

Google ORION SDN Control Plane

ORION is Google's SDN control plane (NSDI'21) for its datacenter (Jupiter) and wide area (B4) networks.

- **Architecture:**
 - **Orion Core:** Contains the Network Information Base (NIB), Topology Manager, and OpenFlow Front End (OFE).
 - **Core Apps:** Routing (intradomain, iBGP) and Traffic Engineering are implemented as applications on top of the ORION core.
 - **Controls:** Uses edge-edge flow-based controls (e.g., CoFlow scheduling) to meet contract SLAs.
- **Management Plane:** Pub-sub distributed microservices in the Orion core; OpenFlow is used for switch signaling/monitoring.
- **Note:** ORION primarily provides **intradomain services** within Google's network.

Google, SDN: The Value of a Logically-Centralized Abstraction

According to Amin Vahdat (Engineering Fellow, VP at Google,

https://www.youtube.com/watch?v=qn7d_iK92TI):

- The ability to design a network where **the location and role of every network element is known ahead of time** is key.
- Configuration shifts from managing individual network elements to managing the **configuration of a logical network**.
- This dramatically simplifies the problem from both a technical perspective and in terms of the **cognitive load on network managers**.

IX. SDN: Selected Challenges and Future

Challenges

- **Hardening the Control Plane:** Must be a dependable, reliable, performance-scalable, and secure distributed system.
- **Robustness to Failures:** Requires leveraging the strong theory of reliable distributed systems for the control plane.
- **Dependability, Security:** Needs to be "baked in" from day one.

- **Mission-Specific Requirements:** Need to support networks and protocols meeting specific needs (e.g., real-time, ultra-reliable, ultra-secure).
- **Internet-Scaling:** Scaling solutions beyond a single Autonomous System (AS).
- SDN is also **critical in 5G cellular networks**.

SDN and the Future of Traditional Network Protocols

The evolution will involve a mix of:

- **SDN-computed** versus **router-computed** forwarding tables.
- **Logically-centralized-computed** versus **protocol-computed** functions.

One could imagine SDN-computed congestion control, where the controller sets sender rates based on congestion levels reported by routers to the controller. The question remains: **How will the implementation of network functionality (SDN versus protocols) evolve?**

Chapter 6: The Link Layer & LANS

I. Fundamentals and Context

Terminology

- **Nodes:** Hosts and routers.
- **Links:** Communication channels that connect **adjacent nodes** along a communication path (e.g., wired, wireless, LANs).
- **Frame:** A Layer-2 packet that **encapsulates** a network-layer datagram.

Responsibility

The link layer has the responsibility of transferring a **datagram** from one node to its **physically adjacent node** over a single link.

Datagram Transfer

A datagram is typically transferred by **different link protocols** over different links along its path (e.g., Wi-Fi on the first link, Ethernet on the next). Each link protocol can provide different services (e.g., some may offer reliable data transfer, others may not).

Transportation Analogy

Networking Component	Transportation Analogy
Tourist	Datagram
Transport Segment	Communication Link (e.g., Princeton to JFK)
Transportation Mode	Link-Layer Protocol (e.g., Limo, Plane, Train)
Travel Agent	Routing Algorithm

II. Link Layer Services

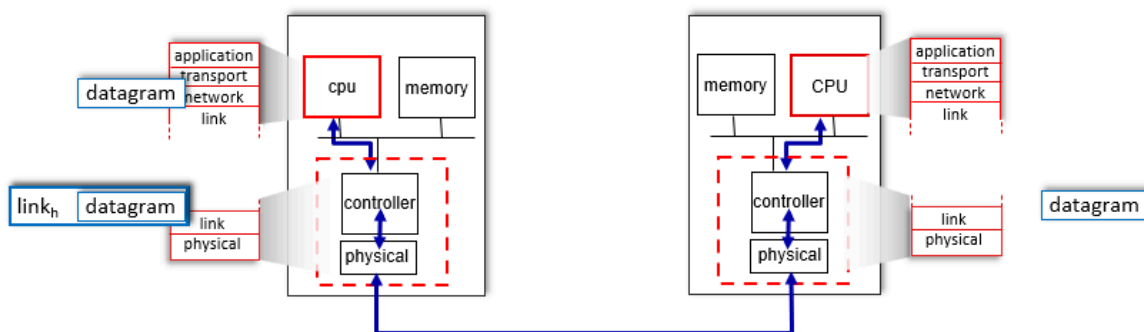
The link layer provides several crucial services:

- **Framing and Link Access:**
 - Encapsulates the datagram into a frame by adding a header and a trailer.
 - Manages **channel access** if the medium is shared.

- Uses **"MAC" addresses** in the frame headers to identify the source and destination (these are **different from IP addresses**).
- **Reliable Delivery between Adjacent Nodes:**
 - Seldom used on links with low bit-error rates (like fiber).
 - Commonly used on wireless links, which have high error rates.
 - *Q: Why both link-level and end-to-end reliability?* Link-level handles errors quickly on high-error links, while end-to-end (TCP) ensures ultimate delivery across the entire path.
- **Flow Control:** Pacing the transmission rate between adjacent sending and receiving nodes.
- **Error Detection:**
 - Detects errors caused by signal attenuation and noise.
 - The receiver detects the error and signals for retransmission or drops the frame.
- **Error Correction:** The receiver can identify and correct bit error(s) without requiring retransmission.
- **Half-Duplex and Full-Duplex:**
 - **Full-duplex:** Nodes can transmit and receive simultaneously.
 - **Half-duplex:** Nodes at both ends can transmit, but **not at the same time**.

Host Link-Layer Implementation

- The link layer is implemented in each host, typically on-chip or in the **Network Interface Card (NIC)**.
- The NIC implements both the link and physical layers and attaches to the host's system buses.
- Implementation is a combination of **hardware, software, and firmware**.
- **Sending Side:** Encapsulates the datagram in a frame and adds error checking, reliable data transfer, and flow control bits.
- **Receiving Side:** Looks for errors, handles reliable data transfer and flow control, extracts the datagram, and passes it to the upper network layer.



III. Multiple Access Links and Protocols

Types of Links

1. **Point-to-Point:** A single link connecting two devices.

- Examples: Point-to-point link between an Ethernet switch and a host; PPP for dial-up access.
- 2. **Broadcast (Shared Wire or Medium):** Multiple nodes share the same physical communication channel.
 - Examples: Old-school Ethernet (shared wire), upstream HFC in cable networks, 802.11 Wi-Fi, 4G/5G, satellite (shared radio).
 - *Analogy:* Humans at a cocktail party (shared air, acoustical).

Multiple Access Protocols (MAC)

- Used for a single shared broadcast channel.
- **Interference/Collision:** Occurs if two or more simultaneous transmissions by nodes are received at the same time.
- **Goal:** A **distributed algorithm** that determines how nodes share the channel (i.e., when a node can transmit).
 - *Challenge:* Communication about channel sharing must use the channel itself (no out-of-band channel for coordination).

Ideal Multiple Access Protocol Desiderata

Given a MAC channel of rate R bps:

1. **Single Active Node:** The node can send at rate R .
2. **M Active Nodes:** Each can send at average rate R/M .
3. **Fully Decentralized:** No special coordinating node; no synchronization of clocks or slots.
4. **Simple.**

MAC Protocols Taxonomy

MAC protocols fall into three broad classes:

1. **Channel Partitioning:**
 - Divides the channel into smaller, non-overlapping "pieces" (time slots, frequency, or code).
 - Allocates a piece to a node for **exclusive use**.
2. **Random Access:**
 - Channel is **not divided**; nodes transmit when they have data, **allowing collisions**.
 - Protocols specify how to detect and "recover" from collisions.
3. **"Taking Turns":**
 - Nodes take turns, but nodes with more data can potentially take longer turns (seeking the best of both worlds).

IV. Channel Partitioning MAC Protocols

These protocols provide efficient and fair sharing at **high load** but are **inefficient at low load** (delay in channel access, only $1/N$ bandwidth allocated even if only 1 node is active).

1. TDMA (Time Division Multiple Access)

- Access is in **"rounds"**.
- Each station gets a fixed-length slot (equal to packet transmission time) in each round.
- Unused slots go idle.

2. FDMA (Frequency Division Multiple Access)

- Channel spectrum is divided into fixed **frequency bands**.
- Each station is assigned a fixed frequency band.
- Unused transmission time in frequency bands goes idle.

V. Random Access Protocols

These protocols are **efficient at low load** (single node can fully utilize the channel) but suffer from **collision overhead at high load**.

When a node has a packet, it transmits at the **full channel data rate** R with no a priori coordination.

1. Slotted ALOHA

Assumptions:

- All frames are the same size.
- Time is divided into equal-size slots (time to transmit one frame).
- Nodes start transmission only at the beginning of a slot.
- Nodes are synchronized, and if 2 nodes transmit, all nodes detect a collision (C).

Operation:

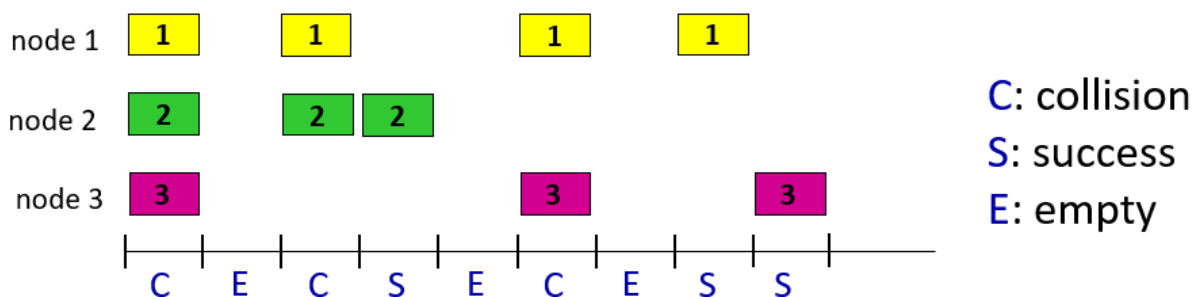
- Node transmits a fresh frame in the next slot.
- **No collision (S)**: Node can send a new frame in the next slot.
- **Collision (C)**: Node retransmits the frame in each subsequent slot with probability p until success.

Pros	Cons
Single active node can continuously transmit at full rate.	Collisions, wasting slots.
Highly decentralized (only slots need to be in sync).	Idle slots (E).
Simple.	Requires clock synchronization.

Efficiency: The long-run fraction of successful slots, maximized as N to infinity, is:

Max Efficiency = $1/e$ approx 0.37

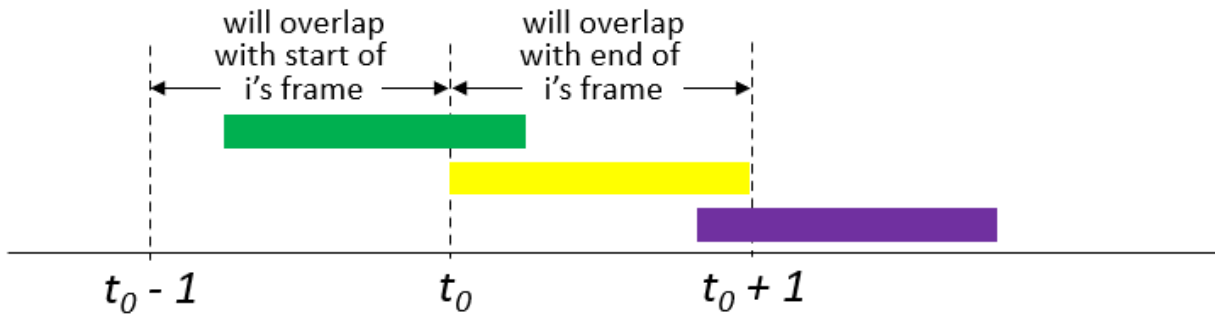
At best, the channel is used for useful transmissions approx 37% of the time.



2. Pure ALOHA (Unslotted)

- Simpler, no synchronization.

- When a frame arrives, it is **transmitted immediately**.
- Collision probability is higher: A frame sent at t_0 collides with any other frame sent in the interval $[t_0-1, t_0+1]$ [Image illustrating a Pure ALOHA collision window, showing that a frame's transmission at t_0 will overlap with any other transmission starting in $[t_0-1, t_0+1]$].
- **Pure ALOHA efficiency**: approx 18% (half that of Slotted ALOHA).



3. CSMA (Carrier Sense Multiple Access)

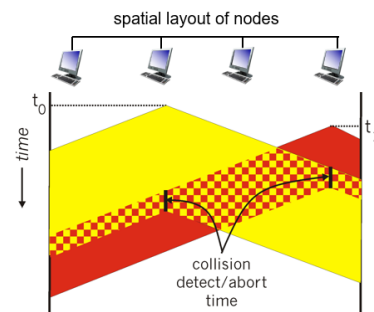
- **Listen Before Transmit:**
 - If the channel is sensed **idle**: Transmit the entire frame.
 - If the channel is sensed **busy**: Defer transmission.
- **Collisions Still Occur:** Due to **propagation delay**, two nodes may not hear each other's just-started transmission, leading to a collision [Image showing two distant nodes sensing an idle channel and starting transmission, resulting in a collision in the middle due to propagation delay].
- The entire packet transmission time is wasted.

4. CSMA/CD (CSMA with Collision Detection)

- **Collision Detection:** Collisions are detected within a short time, and the colliding transmissions are **aborted**, reducing channel wastage.
- **Note:** Collision detection is easy in wired media (e.g., Ethernet) but difficult with wireless media.

Ethernet CSMA/CD Algorithm:

1. Receive datagram, create frame.
2. **Sense Channel:** If idle, start transmission. If busy, wait until idle, then transmit.
3. If the entire frame is transmitted without collision, **done**.
4. If another transmission is detected while sending: **Abort** and send a **jam signal**.
5. After aborting, enter **binary (exponential) backoff**:
 - After the m -th collision, choose K at random from $\{0, 1, 2, \dots, 2^m - 1\}$.
 - Wait $K \cdot 512$ bit times, then return to Step 2. (More collisions means a longer backoff interval).



CSMA/CD Efficiency:

$$\text{Efficiency} = \frac{1}{1 + 5 \frac{t_{prop}}{t_{trans}}}$$

- t_{prop} = max propagation delay between two nodes.
- t_{trans} = time to transmit the max-size frame.
- Efficiency approaches 1 as $t_{prop} \rightarrow 0$ or $t_{trans} \rightarrow \infty$.
- Offers **better performance** than ALOHA and is simple, cheap, and decentralized.

VI. "Taking Turns" MAC Protocols

These protocols aim for the best of both worlds: efficient at low load and fair/efficient at high load.

1. Polling

- A **centralized controller** ("master") invites other nodes ("slaves") to transmit in turn.
- Typically used with "dumb" devices (e.g., Bluetooth).
- **Concerns:** Polling overhead, latency, and the master being a single point of failure.

2. Token Passing

- A control message called the **token** is explicitly passed sequentially from one node to the next.
- A node can only transmit **while holding the token**.
- **Concerns:** Token overhead, latency, and the token itself being a single point of failure.

VII. Cable Access Network: DOCSIS

The cable access network uses a combination of FDM, TDM, and random access, specified by **DOCSIS (Data Over Cable Service Interface Specification)**.

- **Downstream (Cable Headend to Residences):**
 - Multiple downstream (broadcast) **FDM channels** (up to 1.6 Gbps/channel).
 - A single CMTS (Cable Modem Termination System) transmits into these channels.
- **Upstream (Residences to Cable Headend):**
 - Multiple upstream channels (up to 1 Gbps/channel).
 - **Multiple Access:** All users contend (random access) for certain upstream channel time slots; others are assigned **TDM**.
- **Control/Coordination:**
 - A downstream **MAP frame** assigns upstream time slots.
 - Requests for upstream slots (and data) are transmitted using **random access (binary backoff)** in selected minislots [Image illustrating DOCSIS time slots, showing a downstream MAP frame assigning upstream minislots for either contention (random access requests) or TDM (assigned data frames)].

VIII. Summary of MAC Protocols

Class	Mechanism	Examples	Notes
Channel Partitioning	Division by time, frequency, or code	TDMA, FDMA	Efficient at high load, inefficient at low load.
Random Access	Transmit at full rate; collision detection/recovery	ALOHA, S-ALOHA, CSMA, CSMA/CD, CSMA/CA	Efficient at low load, collision overhead at high load.
	Carrier Sensing	CSMA/CD (Ethernet)	Easy in wire, hard in wireless (802.11 uses CSMA/CA).
"Taking Turns"	Centralized control or explicit passing	Polling, Token Passing (Bluetooth, FDDI)	Aims for balance between low-load efficiency and high-load fairness.

MAC Addresses and Address Resolution (ARP)

I. MAC Addresses: Definition and Properties

MAC (Media Access Control) addresses, also known as **LAN**, **physical**, or **Ethernet** addresses, are used for **local** communication.

Feature	IP Address (Network Layer)	MAC Address (Link Layer)
Layer	Layer 3 (Network Layer)	Layer 2 (Link Layer)
Size	32-bit (e.g., 128.119.40.136)	48-bit (for most LANs, e.g., 1A-2F-BB-76-09-AD)
Scope/Function	Used for global network-layer forwarding (routing across the internet).	Used "locally" to get a frame from one interface to another physically-connected interface (within the same subnet).
Format	Dotted Decimal	Hexadecimal (base 16) notation (each numeral represents 4 bits).
Portability	Not portable: Depends on the IP subnet the node is attached to.	Flat address, portable: Can move the interface from one LAN to another.
Source	Assigned by network administrator (depends on subnet).	Burned into NIC ROM by the manufacturer, but sometimes software settable.

Administration	N/A	Administered by the IEEE (Institute of Electrical and Electronics Engineers), which allocates portions of the address space to manufacturers to assure uniqueness.
Analogy	Postal address	Social Security Number

- **Uniqueness:** Every interface on a LAN has a unique 48-bit MAC address.

II. ARP: Address Resolution Protocol

The **Address Resolution Protocol (ARP)** solves the key question: **How to determine an interface's MAC address, knowing its IP address?**

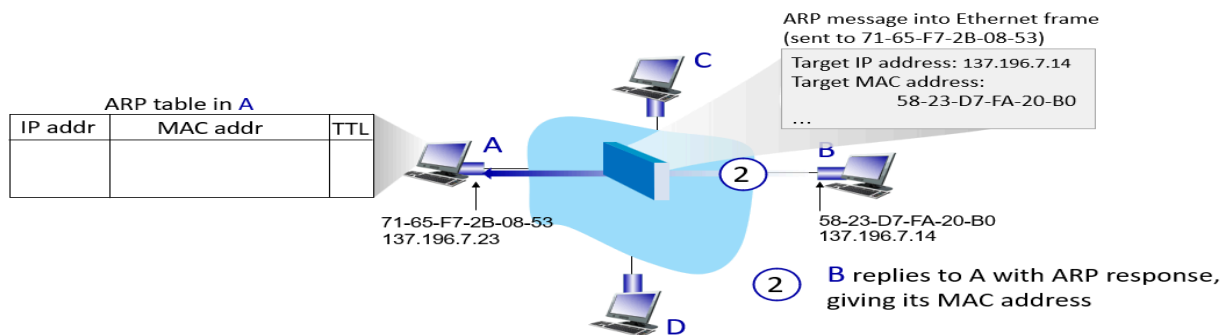
The ARP Table

Each IP node (host, router) on a LAN maintains an **ARP table** in its memory.

- **Contents:** IP/MAC address mappings for some LAN nodes:
< IP address; MAC address; TTL (Time To Live) >
- **TTL:** The time after which the address mapping will be forgotten (typically 20 minutes).

ARP Protocol in Action (A wants to send a datagram to B on the same LAN)

1. **Check ARP Table:** Host A checks its ARP table for B's MAC address. Since it's not found, A initiates ARP.
2. **A Broadcasts ARP Query:** A creates an Ethernet frame containing an ARP query message and **broadcasts** it onto the LAN.
 - **Destination MAC Address:** FF-FF-FF-FF-FF-FF (The broadcast address, ensuring all nodes receive the query).
 - **Contents:** Source MAC (A's MAC), Source IP (A's IP), Target IP address (B's IP). [Image illustrating Host A broadcasting an ARP query with destination MAC FF-FF-FF-FF-FF-FF, and all nodes (B, C, D) receiving it].
3. **B Receives and Replies:** All nodes on the LAN receive the query. Only node B recognizes its own IP address in the query.
4. **B Unicasts ARP Response:** B replies directly (unicasts) to A with an ARP response message, giving its own MAC address.
5. **A Updates Table:** A receives B's reply, adds the B entry (< B's IP; B's MAC; TTL>) into its local ARP table, and then can proceed with the original datagram transmission.



III. Routing to Another Subnet: Addressing Walkthrough

When host A wants to send a datagram to host B, and they are on **different subnets** (connected by router R), the addressing process involves both IP and MAC layers.

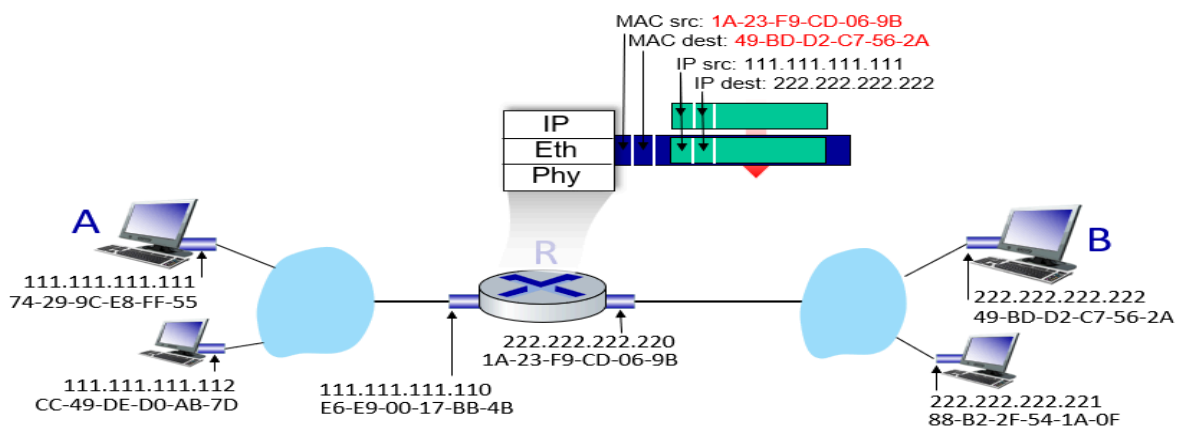
- **Assumptions:** A knows B's IP address, R's IP address (the first-hop router), and R's MAC address (via DHCP and ARP).

Step 1: A to R (First Hop)

- **A creates IP Datagram:** Source IP = A's IP; Destination IP = B's IP(stays same).
- **A creates Link-Layer Frame:** A consults its routing table and sees B is not on the local subnet, so the datagram must go to router R (gateway).
 - **MAC Source:** A's MAC address (e.g., 74-29-9C-E8-FF-55).
 - **MAC Destination:** R's MAC address (e.g., E6-E9-00-17-BB-4B). [Image illustrating the frame sent from A to R, showing MAC source A, MAC destination R, and fixed IP source A, IP destination B].
- The frame is transmitted from A to R.

Step 2: R Processing (Hop-by-Hop)

- **R Receives Frame:** R's interface receives the frame, removes the datagram, and passes it up to the IP layer.
- **R Forwards Datagram:** The IP layer in R uses its routing table to determine the outgoing interface (the one connected to B's subnet). The datagram itself is unchanged (IP source A, IP destination B).
- **R Creates New Link-Layer Frame:** R passes the datagram down to the link layer on the outgoing interface.
 - **MAC Source:** R's outgoing interface MAC address (e.g., 1A-23-F9-CD-06-9B).
 - **MAC Destination:** B's MAC address (e.g., 49-BD-D2-C7-56-2A).
- The new frame is transmitted from R toward B.



Step 3: R to B (Final Hop)

- **B Receives Frame:** B's NIC receives the frame, extracts the IP datagram (which has destination B's IP).

- **B Processes Data:** B passes the datagram up its protocol stack to the IP layer and eventually to the transport/application layer.

IV. Ethernet: The Dominant LAN Technology

Ethernet is the "dominant" wired LAN technology because:

- It was the **first widely used** LAN technology.
- It is **simpler and cheaper**.
- It has **kept up with the speed race** (10 Mbps up to 400 Gbps).

Ethernet: Physical Topology

- **Bus (Through mid-90s):** All nodes shared the same cable, meaning all nodes were in the **same collision domain** (could collide with each other).
- **Switched (Prevails Today):** An **active link-layer 2 switch** is in the center. Each "spoke" runs a **separate** Ethernet protocol, and nodes **do not collide** with each other.

Ethernet: Unreliable and Connectionless

- **Connectionless:** No handshaking (setup) between sending and receiving NICs.
- **Unreliable:** The receiving NIC **does not send ACKs or NAKs** to the sending NIC.
 - Data in dropped frames is recovered **only if the initial sender uses a higher-layer Reliable Data Transfer (rdt) protocol** (like TCP); otherwise, the dropped data is lost.
- **MAC Protocol:** Uses **unslotted CSMA/CD with binary backoff**. Binary exponential wait time on collision detections, doubled every random timeout

802.3 Ethernet Standards

- There are many different standards with common MAC protocol and frame format.
- They vary in **speed** (2 Mbps to 40 Gbps and beyond) and **physical layer media** (fiber, cable).

V. Ethernet Frame Structure

Ethernet frame structure

sending interface encapsulates IP datagram (or other network layer protocol packet) in **Ethernet frame**



preamble:

- used to synchronize receiver, sender clock rates
- 7 bytes of 10101010 followed by one byte of 10101011

Field	Size	Purpose
Preamble	8 bytes	Used to synchronize receiver and sender clock rates. (7 bytes of 10101010 followed by 1 byte of 10101011).
Dest Address	6 bytes	Destination MAC address.
Source Address	6 bytes	Source MAC address.
Type	2 bytes	Indicates the higher-layer protocol (e.g., IP, Novell IPX, AppleTalk). Used to demultiplex the packet up the stack at the receiver.
Data (Payload)	46 - 1500 bytes	Encapsulates the IP datagram (or other network-layer packet).
CRC (Cyclic Redundancy Check)	4 bytes	Error detection field. If an error is detected at the receiver, the frame is dropped .

- **Frame Processing:** If an adapter receives a frame with a matching destination address (or a broadcast address, like for ARP), it passes the data to the network layer protocol. Otherwise, the adapter discards the frame.

I. Ethernet Switch Fundamentals

An **Ethernet Switch** is a **link-layer device** that takes an active role in network communication.

- **Operation:** It is a **store-and-forward** device that examines an incoming frame's MAC address and **selectively forwards** the frame to one or more outgoing links.
 - If a frame is to be forwarded onto a segment, the switch uses **CSMA/CD** to access that specific segment.
- **Transparency:** Hosts are **unaware** of the presence of switches.
- **Plug-and-Play (Self-Learning):** Switches do not need to be configured; they **self-learn** the network topology.

Multiple Simultaneous Transmissions

- Hosts have a **dedicated, direct connection** to the switch.
- Switches **buffer packets**.
- The Ethernet protocol is used on each incoming link, resulting in:
 - **No collisions** (full duplex is possible).
 - Each link is its **own collision domain**.
- **Simultaneous Forwarding:** Multiple transmissions can occur simultaneously (e.g., A-to-A' and B-to-B' can transmit without collisions). However, transmissions that share the same outgoing link (e.g., A-to-A' and C-to-A') cannot happen simultaneously.

II. Switch Forwarding (Switch Table)

Switch Table

Each switch maintains a **switch table** (which looks like a routing table) to determine where to forward frames.

- **Entry Format:** < MAC address of host; interface to reach host; time stamp (TTL) >
- **Goal:** The table tells the switch, for example, that host A' is reachable via interface 4.

Self-Learning

The switch learns which hosts can be reached through which interfaces.

- When a frame is received, the switch **"learns" the location of the sender** (the incoming LAN segment).
- It records the **sender MAC address/location pair** in its switch table.

Frame Filtering/Forwarding Process

When a frame is received at a switch:

1. **Record:** Record the incoming link and the MAC address of the sending host (for self-learning).
2. **Index:** Index the switch table using the **MAC destination address**.
3. **If entry found for destination:**
 - If the destination is on the segment from which the frame arrived, **drop the frame (Filtering)**.
 - Else, **forward the frame** on the interface indicated by the entry (**Selective Forwarding**).
4. **Else (entry not found): Flood** the frame (forward on all interfaces except the arriving interface).
 - **Example (Unknown Destination):** If the destination location is unknown, the switch **floods** the frame.
 - **Example (Known Destination):** If the destination location is known, the switch **selectively sends** the frame on just one link.

Interconnecting Switches

Self-learning works seamlessly when multiple switches are connected together (e.g., S1, S2, S3, S4). The learning process is exactly the same as in a single-switch case, allowing the network to scale.

III. Switches vs. Routers

Feature	Switches (Link-Layer Device)	Routers (Network-Layer Device)
Layer	Link-layer (Layer 2)	Network-layer (Layer 3)
Header Examined	Link-layer headers (MAC addresses)	Network-layer headers (IP addresses)
Forwarding Table	Switch Table (uses MAC addresses)	Forwarding Table (uses IP addresses)

Table Computation	Learns forwarding table entries using flooding and self-learning (Plug-and-Play).	Computes forwarding table entries using routing algorithms (e.g., OSPF, BGP).
--------------------------	---	---

IV. Virtual LANs (VLANs)

Motivation

As LANs scale, the size of a **single broadcast domain** becomes problematic:

- **Scaling:** All Layer-2 broadcast traffic (ARP, DHCP, unknown MAC) must cross the entire LAN, leading to **efficiency, security, and privacy issues**.
- **Administrative Issues:** If a user moves their office (physical attachment) but needs to remain logically attached to their original department's network (e.g., a CS user moving to an EE office).

Port-Based VLANs

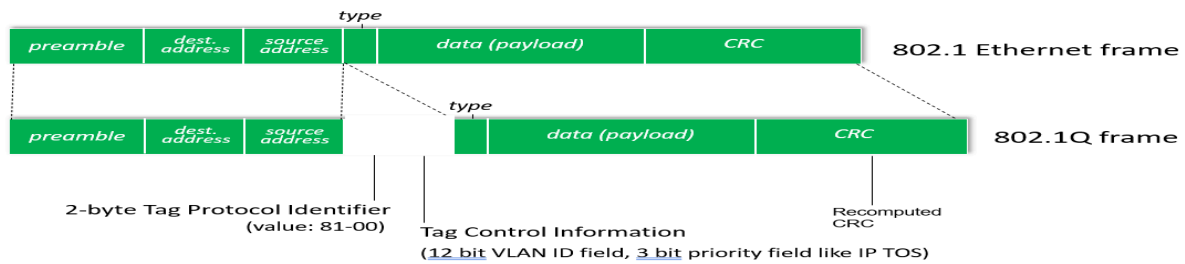
- A **Virtual Local Area Network (VLAN)** allows switch(es) to be configured to define multiple virtual LANs over a single physical infrastructure. Logically split one LAN into multiple broadcast domains
- **Port-Based VLAN:** Switch ports are grouped (via switch management software) so that a single physical switch operates as **multiple virtual switches**.
 - **Traffic Isolation:** Frames to/from ports 1-8 (EE VLAN) can only reach other ports in the EE VLAN.
- **Membership:** VLANs can also be defined based on the MAC addresses of endpoints, not just the switch port. Ports can be dynamically assigned.
- **Forwarding between VLANs:** This is done via **routing** (just as if they were separate physical switches). Vendors often sell combined switches/routers for this purpose.

VLANs Spanning Multiple Switches

- **Trunk Port:** A port that carries frames **between VLANs** defined over multiple physical switches.
- **VLAN ID:** Frames forwarded within a VLAN between switches cannot be vanilla 802.1 Ethernet frames; they **must carry VLAN ID info**.
- **802.1Q Protocol:** This protocol adds/removes additional header fields for frames forwarded between trunk ports.

802.1Q VLAN Frame Format

802.1Q VLAN frame format



The 802.1Q frame adds a 4-byte header between the Source Address and the original Type field.

- **Tag Protocol Identifier (TPI):** 2 bytes (Value: 81-00). Identifies the frame as an 802.1Q VLAN frame.
- **Tag Control Information (TCI):** 2 bytes, containing:
 - **12-bit VLAN ID field:** Identifies the VLAN.
 - **3-bit Priority field** (like IP TOS).
- The **CRC is recomputed** after the tag is inserted.

V. EVPN (VXLANS): Ethernet VPNs

EVPN (or **VXLAN**) is a tunneling scheme used to **overlay Layer 2 networks on top of Layer 3 networks** over an existing infrastructure, effectively "stretching" a Layer 2 network across disparate sites (e.g., datacenters in Sunnyvale and Bangalore).

- **Mechanism:** Layer-2 Ethernet switches are logically connected using an IP network as the **underlay**.
- **Encapsulation:** Ethernet frames are carried within **IP datagrams** between sites.
- **VXLAN Tunnel Endpoints (VTEP):** Devices (like switches/routers) at each site that encapsulate the Ethernet frame inside a UDP segment, which is then placed in an IP datagram for transport across the Internet (the tunnel). [Image illustrating an Ethernet frame encapsulated in a UDP segment, which is placed in an IP datagram, for transport between two VTEPs across the Internet].

VI. Datacenter Networks (DCNs)

Datacenters contain \$10\$'s to \$100\$'s of thousands of closely coupled hosts, serving massive numbers of clients for e-business, content-servers, search, and data mining.

- **Challenges:** Reliability, managing/balancing load, and avoiding processing, networking, and data bottlenecks.

Datacenter Network Elements (Three-Tier Architecture)

1. **Border Routers:** Connect the datacenter to the outside Internet.
2. **Tier-1 (Core) Switches:** Connect to Tier-2 switches below.
3. **Tier-2 (Aggregation) Switches:** Connect to TOR switches below.
4. **Top of Rack (TOR) (Access) Switches:** One per rack, providing high-speed Ethernet to the server blades.
5. **Server Racks:** Contain \$20-40\$ server blades (hosts).

- **Modern Topology:** Many modern DCNs use a **Two-layer leaf/spine** interconnection network (e.g., Facebook F16) instead of the traditional three-tier structure.

Datacenter Networks: Multipath

- DCNs feature **rich interconnection** among switches and racks.
- This provides **increased throughput** (multiple routing paths possible) and **increased reliability** via redundancy. [Image highlighting two disjoint paths between two server racks in a rich, interconnected datacenter topology].

Datacenter Networks: Application-Layer Routing

- A **Load Balancer** sits between the Internet and the datacenter core.
- **Function:** It receives external client requests and directs the workload to specific servers within the datacenter (application-layer routing). It then returns results to the external client, **hiding the data center internals**.

Datacenter Networks: Protocol Innovations

SDN is widely used within/among organizations' datacenters for routing and management.

Layer	Innovation	Description
Link Layer	RoCE	Remote DMA (RDMA) over Converged Ethernet.
Transport Layer	ECN (Explicit Congestion Notification)	Used in transport-layer congestion control (e.g., DCTCP, DCQCN).
Routing, Management	SDN	Google's ORION SDN control plane is used for internal datacenter and wide-area networks.

- **Google ORION SDN:**
 - Routing (intradomain, iBGP) and traffic engineering are implemented as **applications** on top of the ORION core.
 - It manages edge-edge flow-based controls (e.g., CoFlow scheduling) to meet contract SLAs.
 - *Note:* In this model, traditional routing protocols may be absent, and congestion control is partially managed by SDN rather than by traditional protocols.